

Estructuras de Datos

Clase 5 – Implementación de Pilas y Colas con Nodos Enlazados



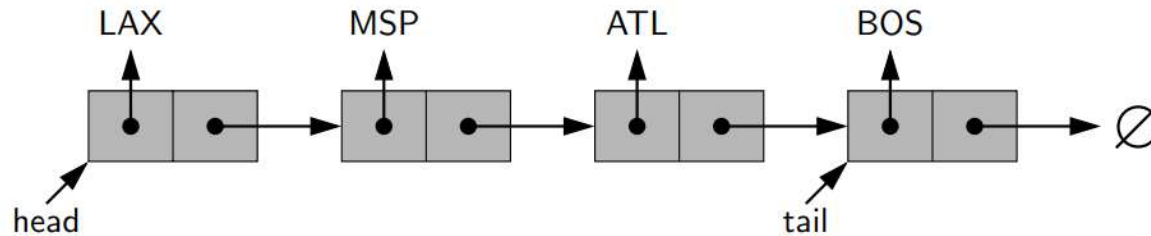
Dr. Sergio A. Gómez
<http://cs.uns.edu.ar/~sag>



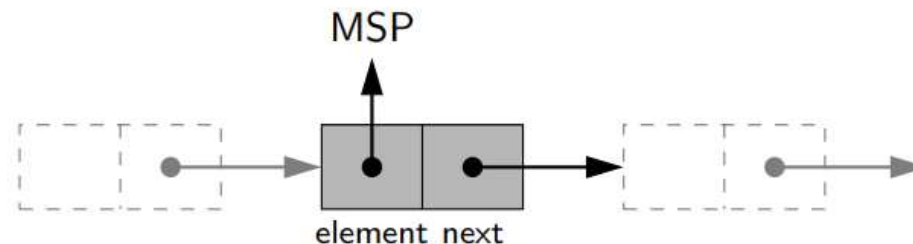
Departamento de Ciencias e Ingeniería de la Computación
Universidad Nacional del Sur
Bahía Blanca, Argentina

Secuencias con nodos enlazados

- La secuencia [LAX, MSP, ATL, BOS] se representa con:



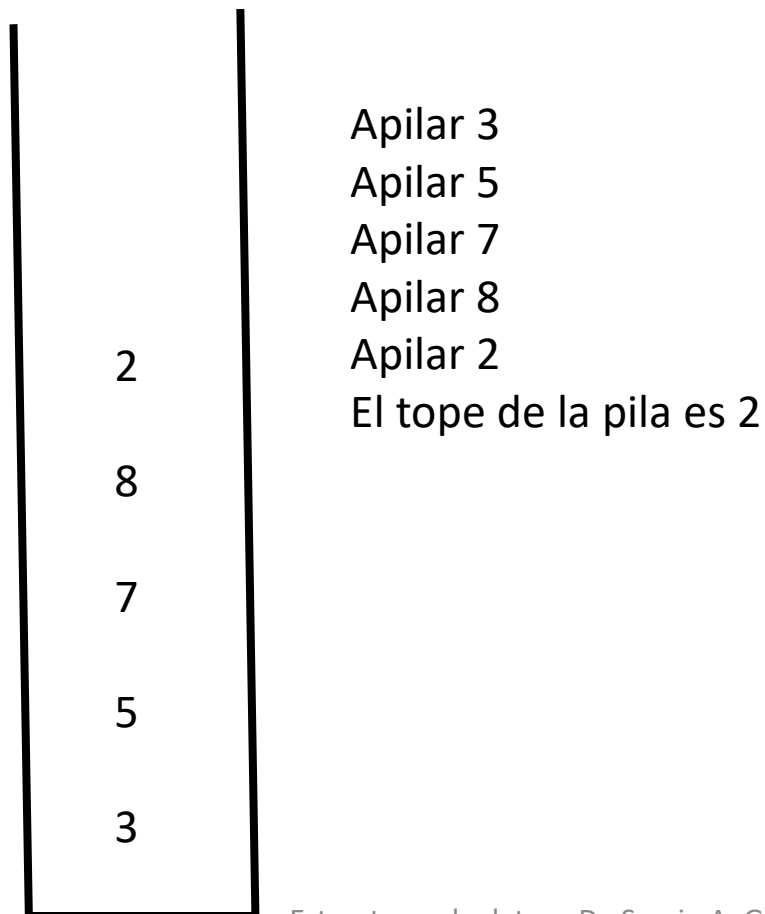
- Hay 2 campos *head* (para el primer elemento) y *tail* (para el último).
- Cada nodo conoce su elemento o rótulo (*element*) y el nodo siguiente en la secuencia (*next*) y se representa como:



- El símbolo $\emptyset$ indica que el siguiente es ninguno y, en Java, se representa con *null*.

Repaso: Tipo de dato abstracto Pila

Pila: Colección lineal de objetos actualizada en un extremo llamado *tope* usando una política LIFO (last-in first-out, el primero en entrar es el último en salir).



Repaso: Interfaz para el TDA Pila

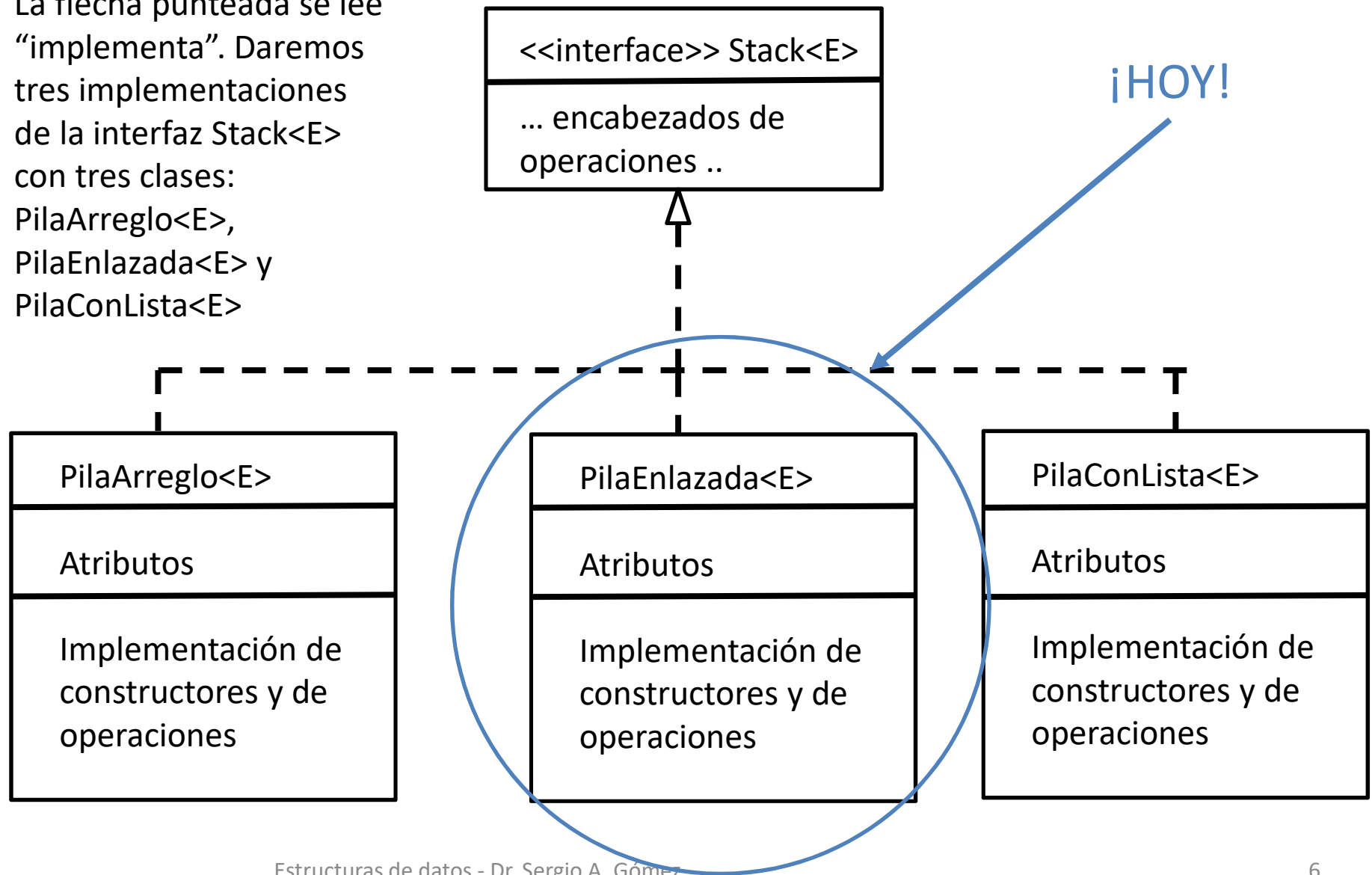
```
public interface Stack<E> {  
    // Inserta item en el tope de la pila.  
    public void push( E item );  
  
    // Retorna true si la pila está vacía y falso en caso contrario.  
    public boolean isEmpty();  
  
    // Elimina el elemento del tope de la pila y lo retorna.  
    // Produce un error si la pila está vacía.  
    public E pop();  
  
    // Retorna el elemento del tope de la pila y lo retorna.  
    // Produce un error si la pila está vacía.  
    public E top();  
  
    // Retorna la cantidad de elementos de la pila.  
    public int size();  
}
```

Implementaciones de pilas

- 1) Con un arreglo (en clase 3):
 - 1) Fácil de implementar y testear
 - 2) Es necesario indicar el tamaño máximo en el constructor
 - 3) La ED es acotada y genera problemas cuando se llena el arreglo para aumentar su tamaño
- 2) Con una estructura de nodos enlazados (hoy):
 - 1) Un poco más difícil de implementar y testear
 - 2) No es necesario indicar el tamaño en el constructor
 - 3) La estructura es no acotada, i.e. puede crecer indefinidamente (mientras haya memoria)
- 3) Con un adaptador en términos de lista (cuando demos el TDA Lista)

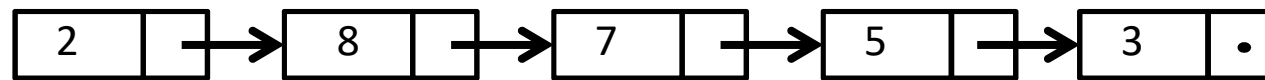
Diagrama UML del diseño

La flecha punteada se lee “implementa”. Daremos tres implementaciones de la interfaz Stack<E> con tres clases: PilaArreglo<E>, PilaEnlazada<E> y PilaConLista<E>



Implementación con nodos enlazados

Utilizaremos una estructura de nodos enlazados. La pila mostrada se representará con nodos (celdas) enlazados:



tope
(top)

Notas:

La pila conoce sólo el nodo *cabeza/tope*, que corresponde al tope o último dato apilado.

Todas las operaciones son eficientes pues la estructura es no acotada y no hay casos especiales porque nunca se llena.

Es más difícil de codificar, depurar y mantener pero se gana en flexibilidad.

Se puede almacenar el tamaño como un atributo más para evitar recorrer la estructura para computar `size()`.

2

8

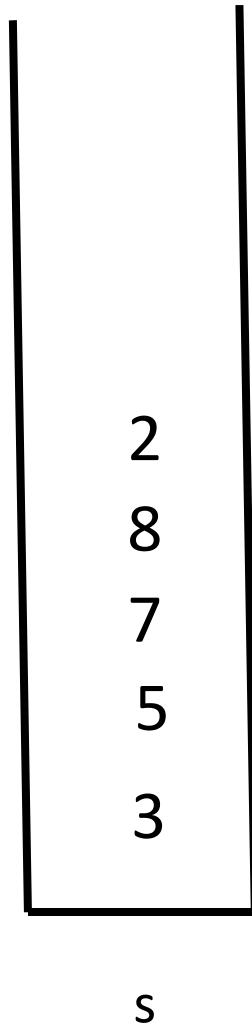
7

5

3

S

Pila: Implementación con nodos enlazados



// Archivo: PilaEnlazada.java

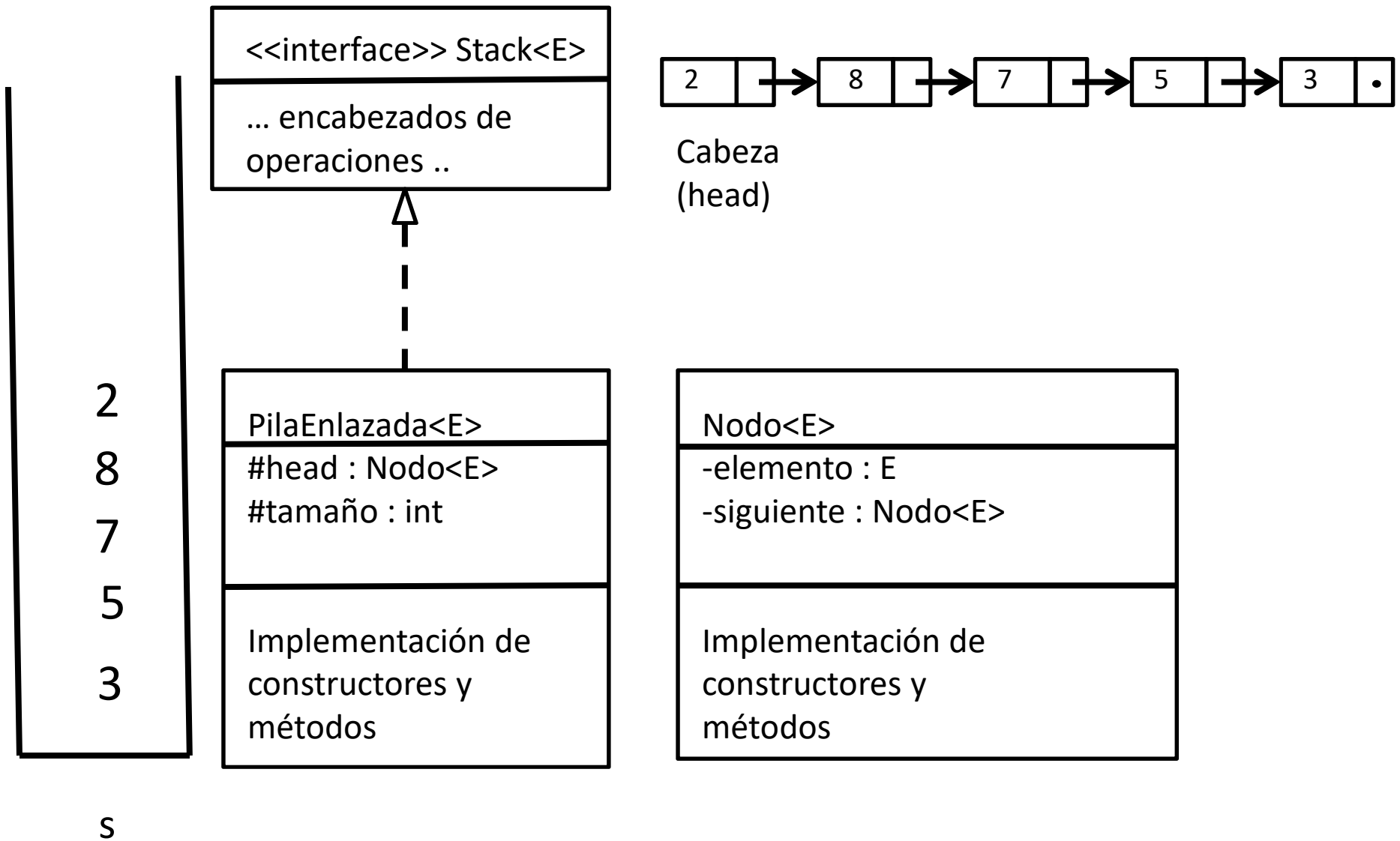
```
public class PilaEnlazada<E> implements Stack<E> {  
    ...  
}
```

// Archivo: AplicacionUsaPilaEnlazada.java

```
public class AplicacionUsaPilaEnlazada {  
    public static void main( String [] args ) {  
        Stack<Integer> s = new PilaEnlazada<Integer>();  
        s.push( 3 );  
        s.push( 5);  
        s.push( 7 );  
        s.push( 8 );  
        s.push( 2 );  
    }  
}
```

Ahora la aplicación debe invocar el constructor de la nueva clase pero como la variable s es de tipo Stack el resto de la implementación de main no cambia.

Implementación de pilas con nodos enlazados



Pila: Implementación con nodos enlazados

```
public class Nodo<E> {
    private E elemento;
    private Nodo<E> siguiente;

    // Constructores:
    public Nodo( E item, Nodo<E> sig )
    { elemento=item; siguiente=sig; }
    public Nodo( E item ) { this(item,null); }

    // Setters:
    public void setElemento( E elemento )
    { this.elemento = elemento; }

    public void setSiguiente( Nodo<E> siguiente )
    { this.siguiente = siguiente; }

    // Getters:
    public E getElemento() { return elemento; }

    public Nodo<E> getSiguiente() { return siguiente; }
}
```

```
public class PilaEnlazada<E>
    implements Stack<E> {
    protected Nodo<E> head;
    protected int tamaño;

    ....
}
```

Nota: La clase Nodo es *recursiva*, ya que los nodos están definidos en términos de nodos pues el campo siguiente hace que nodo conozca a otro nodo.

Pila con nodos enlazados: Constructor

El constructor crea una pila vacía.

Recordemos que:

```
Public class PilaEnlazada<E>
    implements Stack<E> {
    protected Nodo<E> head;
    protected int tamaño;

    ....
}
```

Constructor:

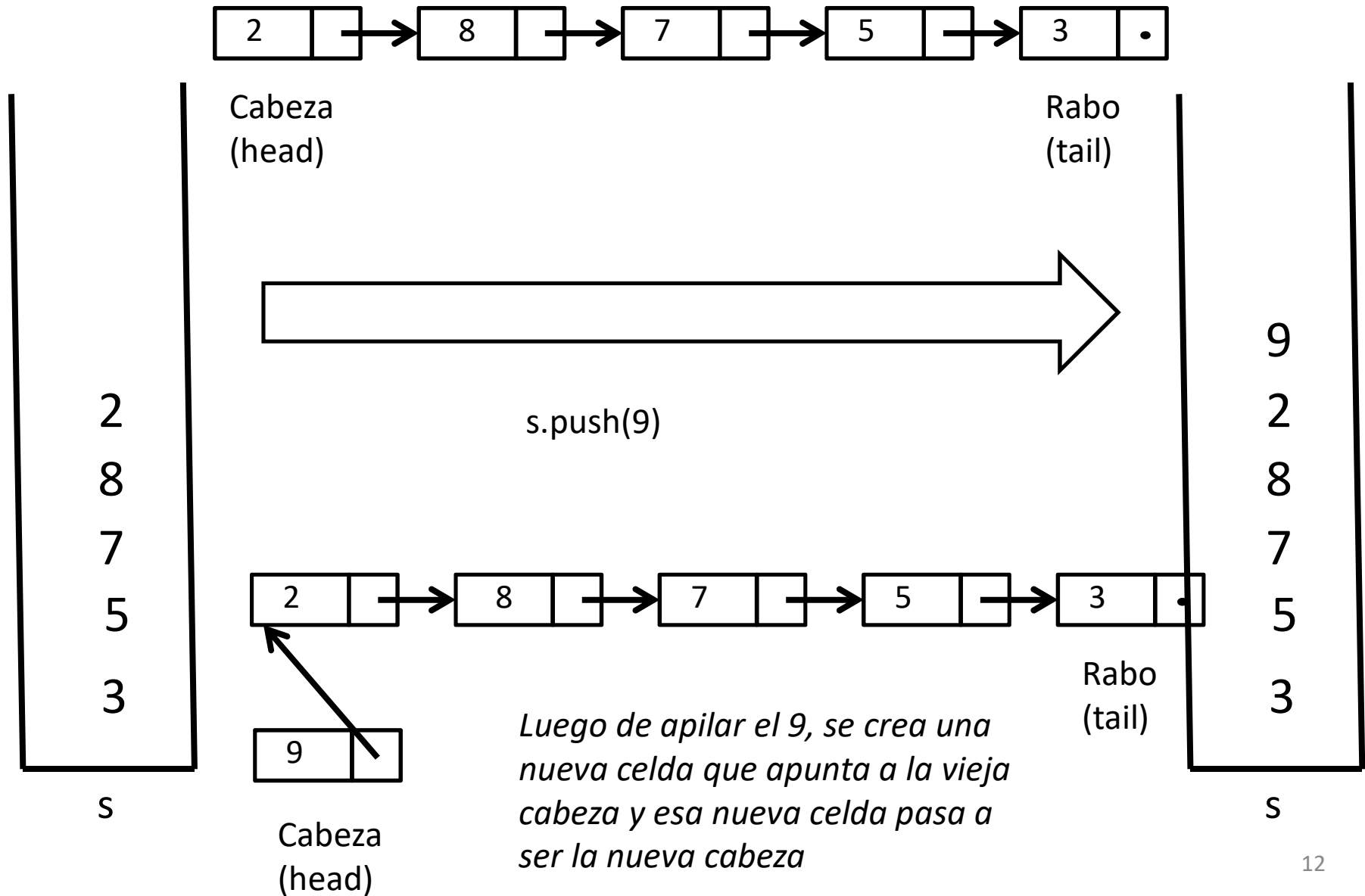
```
PilaEnlazada() {
    head = null
    tamaño = 0
}
```

Un caso de prueba para validar el comportamiento del constructor y la operación isEmpty puede ser:

```
assert (new PilaEnlazada<E>()).isEmpty() == true;
```

S

Apilar



Apilar

```
Public class PilaEnlazada<E>
    implements Stack<E> {
    protected Nodo<E> head;
    protected int tamaño;

    ....
}
```

```
push(item : E)
{
    Nodo<E> aux = new Nodo<E>(item )
    aux.setElemento( item )
    aux.setSiguiente( head )
    head = aux
    tamaño = tamaño + 1
}
```

PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

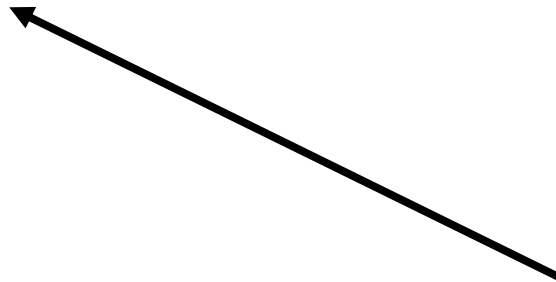
push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
s.push( 7 );
```



¿Cómo es la traza de ejecución de este código?

<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

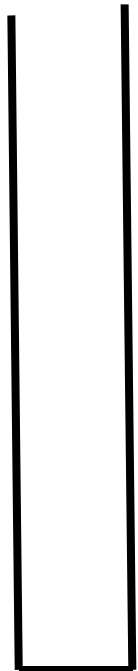
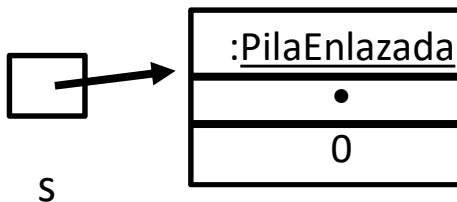
Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
```

Al ejecutar el constructor se crea un objeto de tipo PilaArreglo<Integer> y su referencia se almacena en s.

La pila conceptual es la que veremos a la derecha.



S

PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

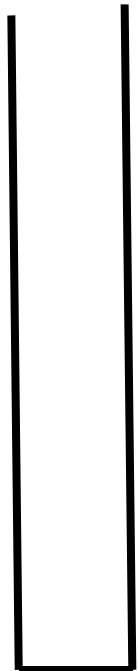
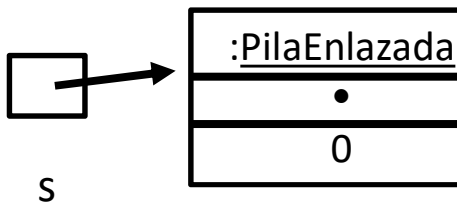
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
```

Ahora queremos apilar un 3



S

<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

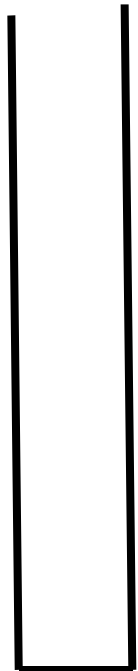
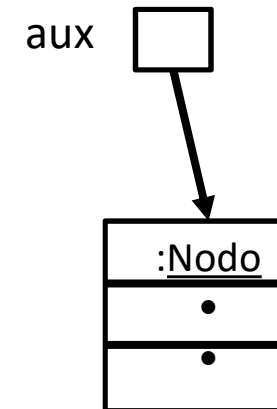
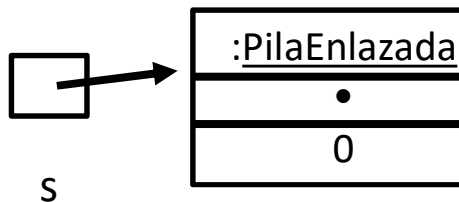
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
```

Ejecutamos push.1: se crea un nodo y se apunta con *aux*



S

PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

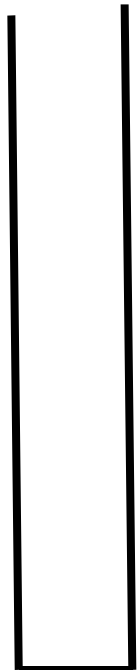
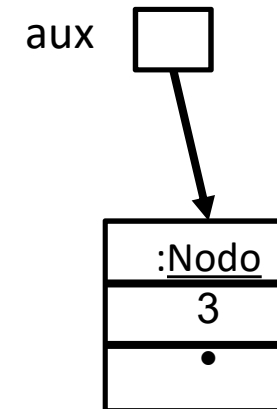
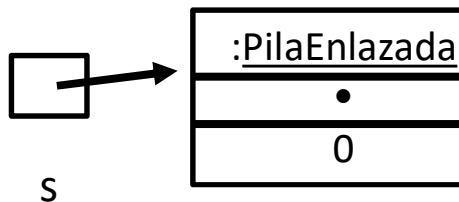
Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

Stack<Integer> s =

```
    new PilaEnlazada<Integer>();
s.push( 3 );
```

Ejecutamos push.2: asignamos 3 en campo *elemento* de *aux*



PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

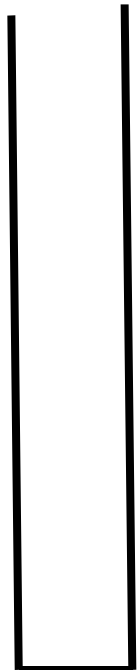
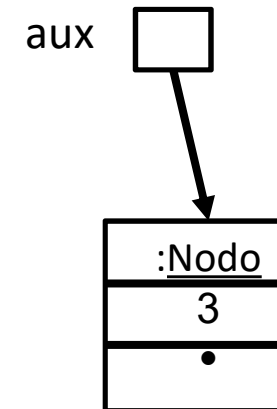
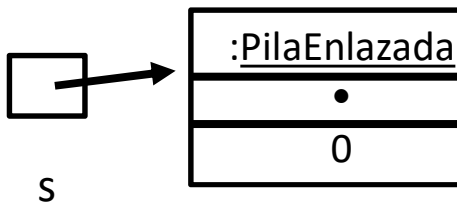
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
```

Ejecutamos push.3:
Se asigna null en null



S

<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

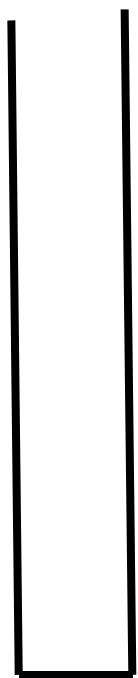
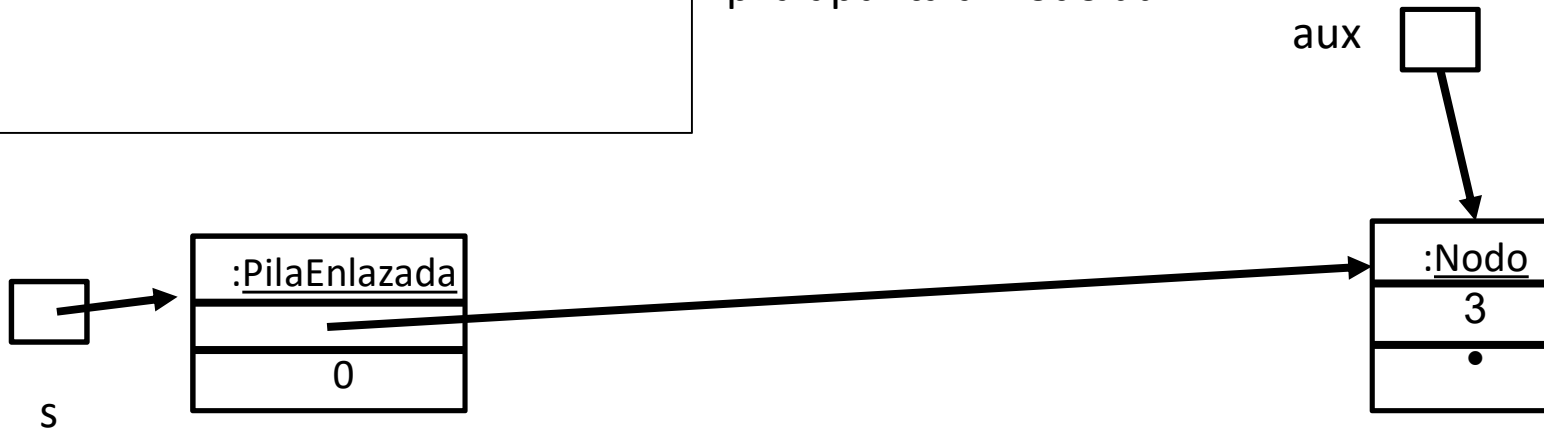
Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

Stack<Integer> s =

```
    new PilaEnlazada<Integer>();
s.push( 3 );
```

Ejecutamos push.4: el campo *head* de la pila apunta al nodo *aux*



PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

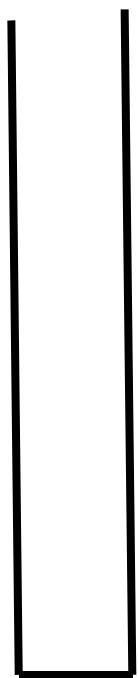
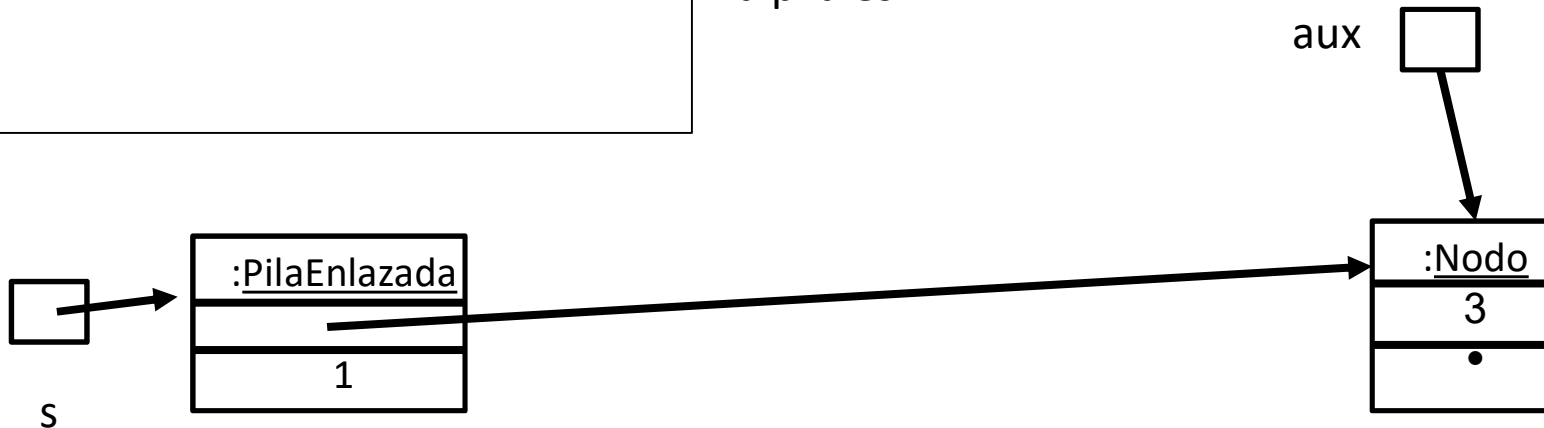
Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

Stack<Integer> s =

```
    new PilaEnlazada<Integer>();
s.push( 3 );
```

Ejecutamos push.5: Ahora el *tamaño* de la pila es 1



S

<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

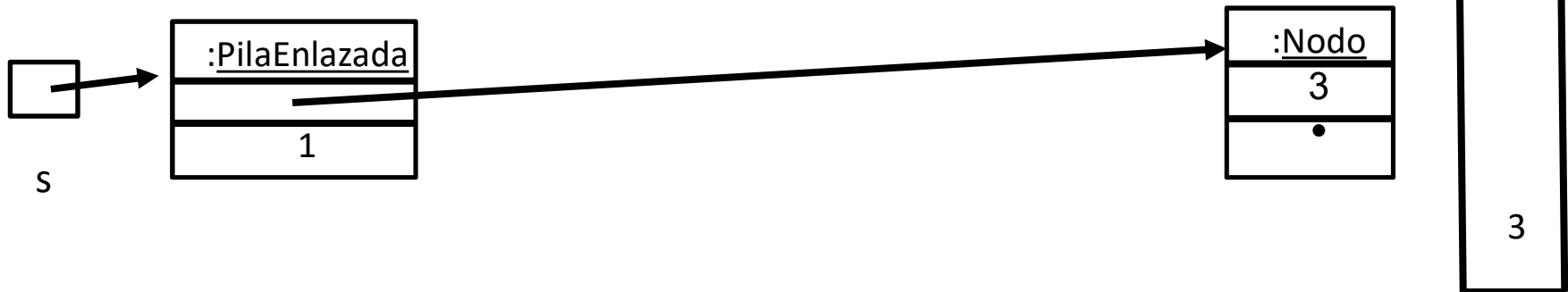
```
{   head = null;   tamaño = 0   }
```

Stack<Integer> s =

```
    new PilaEnlazada<Integer>();
s.push( 3 );
```

Retornamos y aux desaparece porque es una variable local pero el nodo permanece porque está en la memoria dinámica.

Quedó la pila de la derecha.



S

PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

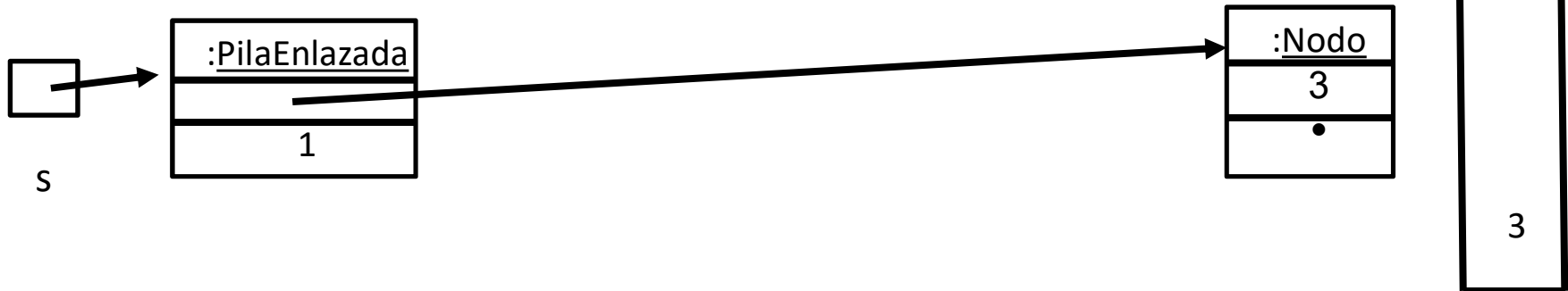
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
```

Ahora queremos apilar un 5.



S

<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

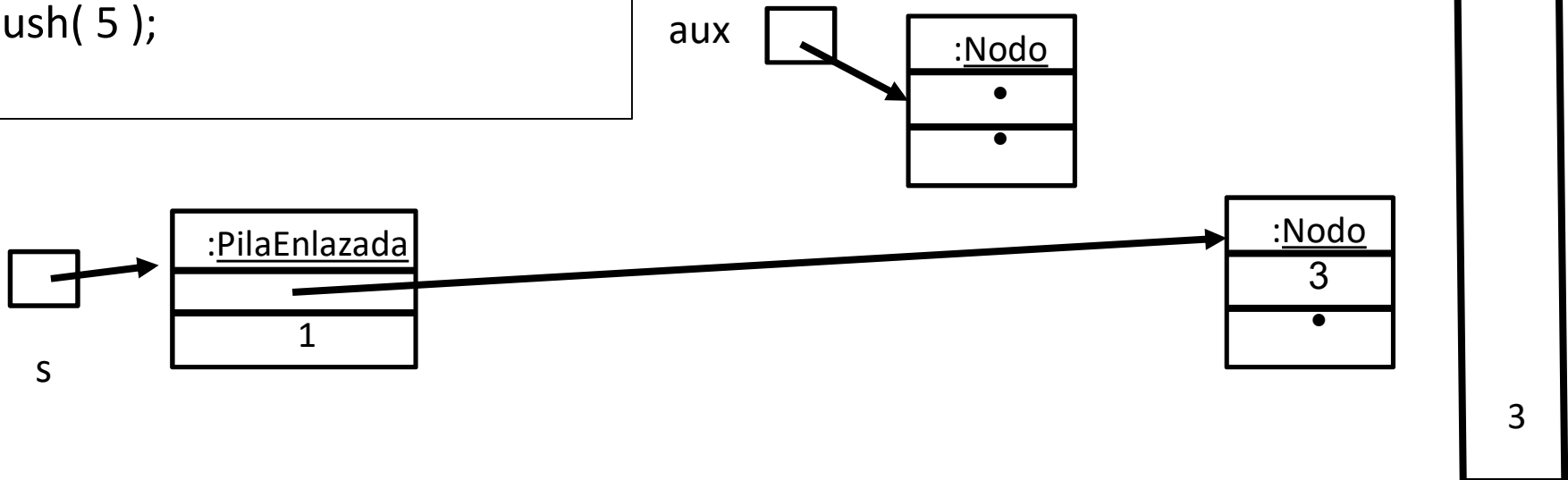
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
```

Ejecutamos push.1: Se crea un nodo nuevo



<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

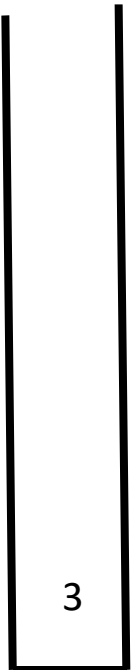
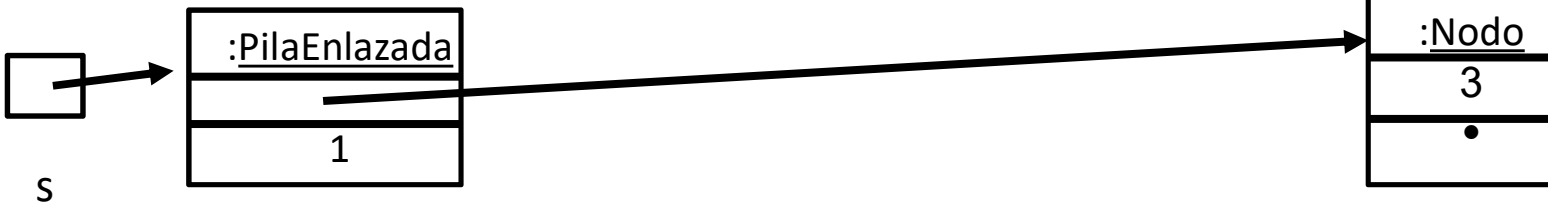
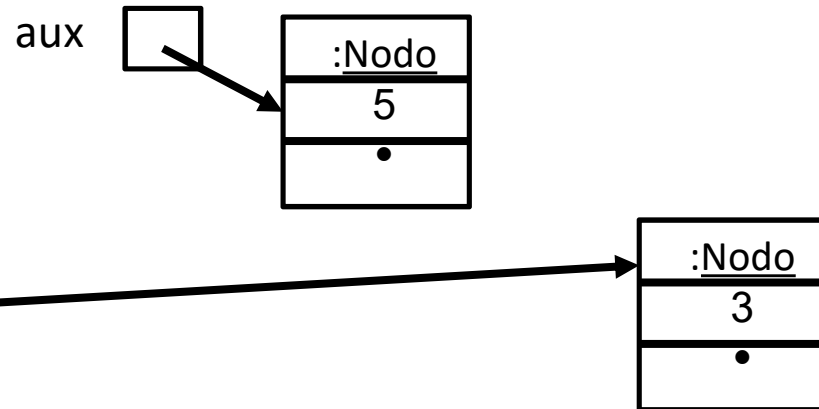
Stack<Integer> s =

new PilaEnlazada<Integer>();

s.push(3);

s.push(5);

Ejecutamos push.2: Almacenamos el *item* en el nodo *aux*



PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

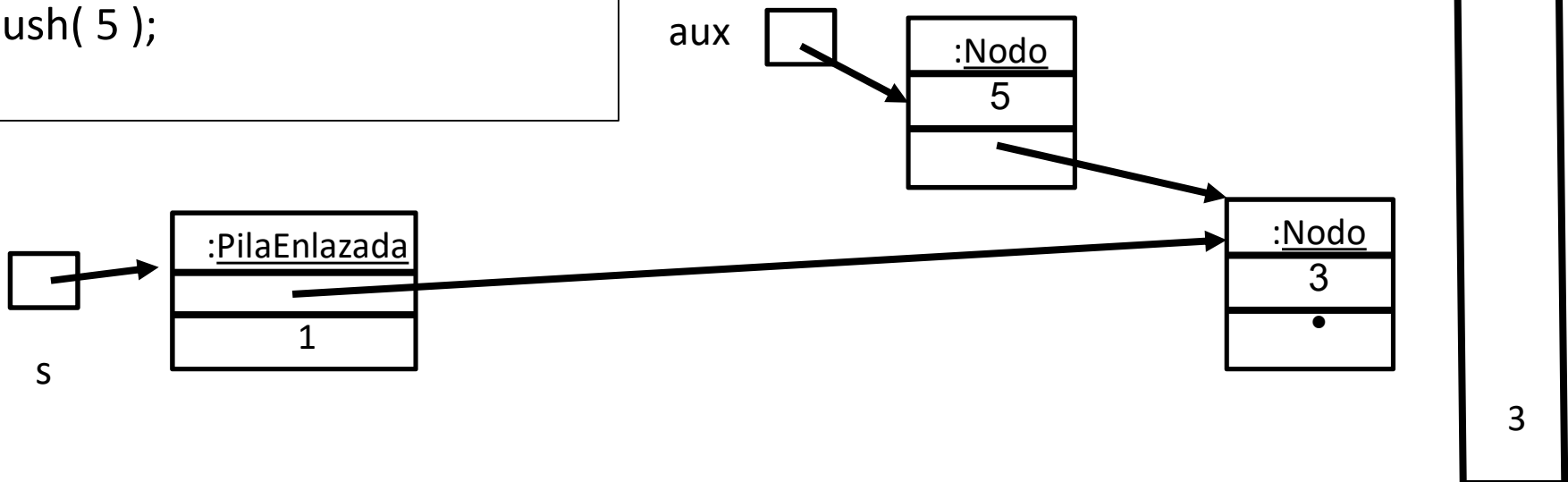
Stack<Integer> s =

new PilaEnlazada<Integer>();

s.push(3);

s.push(5);

Ejecutamos push.3: El siguiente de *aux* es *head*



<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

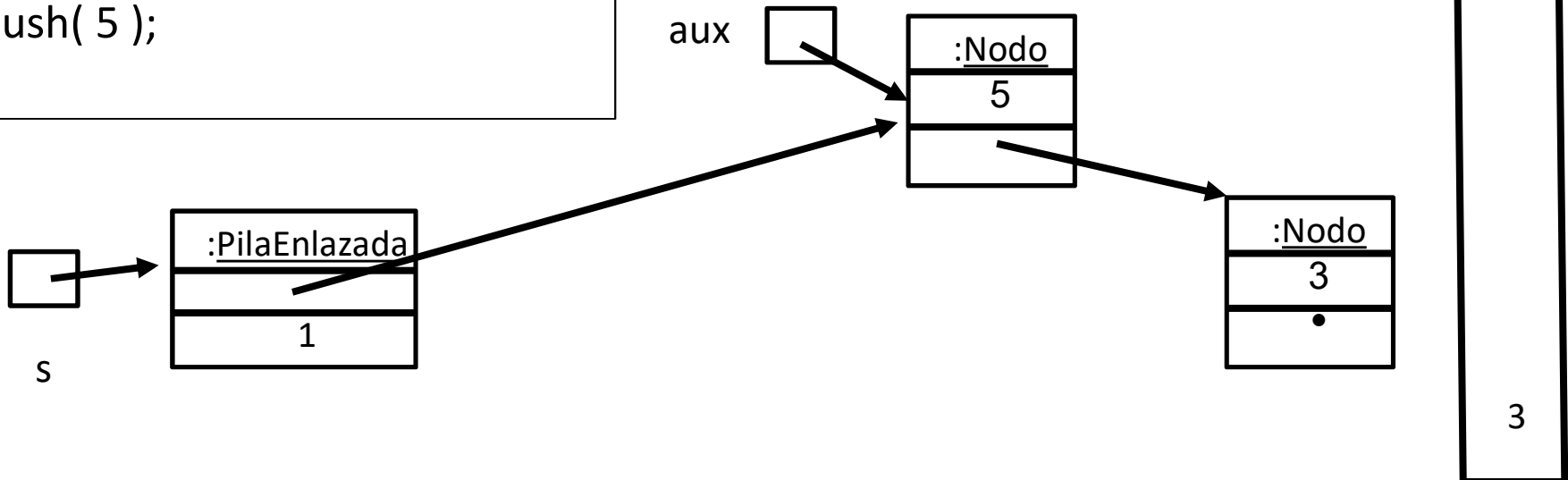
Stack<Integer> s =

new PilaEnlazada<Integer>();

s.push(3);

s.push(5);

Ejecutamos push.4: Ahora *head* apunta al nodo apuntado por *aux*



s

<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

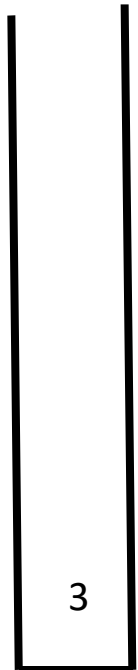
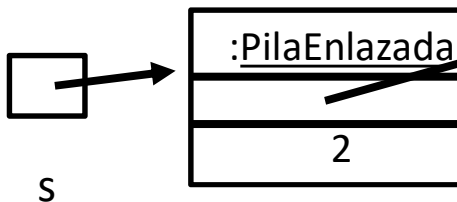
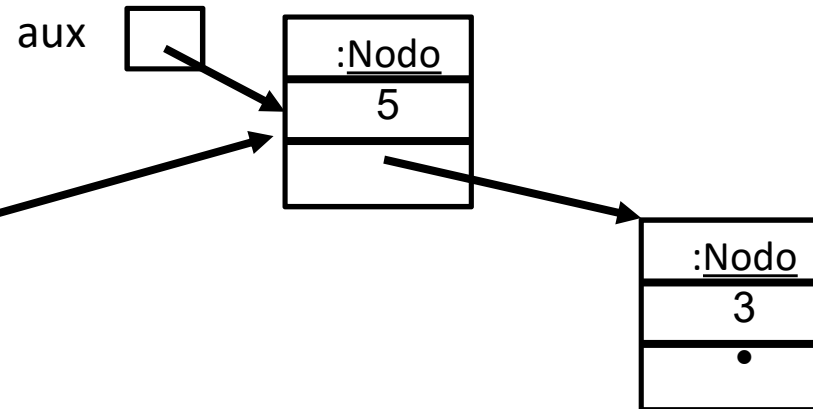
Stack<Integer> s =

new PilaEnlazada<Integer>();

s.push(3);

s.push(5);

Ejecutamos push.5: Aumentamos el tamaño a 2



s

<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

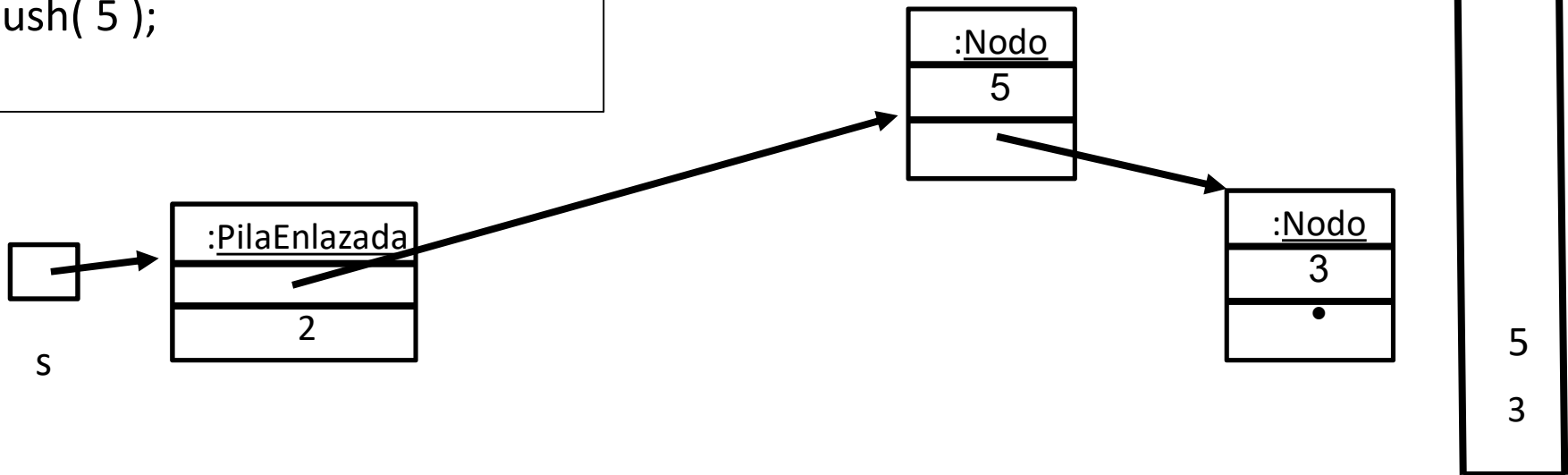
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
```

Retornamos: *aux* se destruye pero no el nodo nuevo y la pila tiene un nuevo elemento



<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

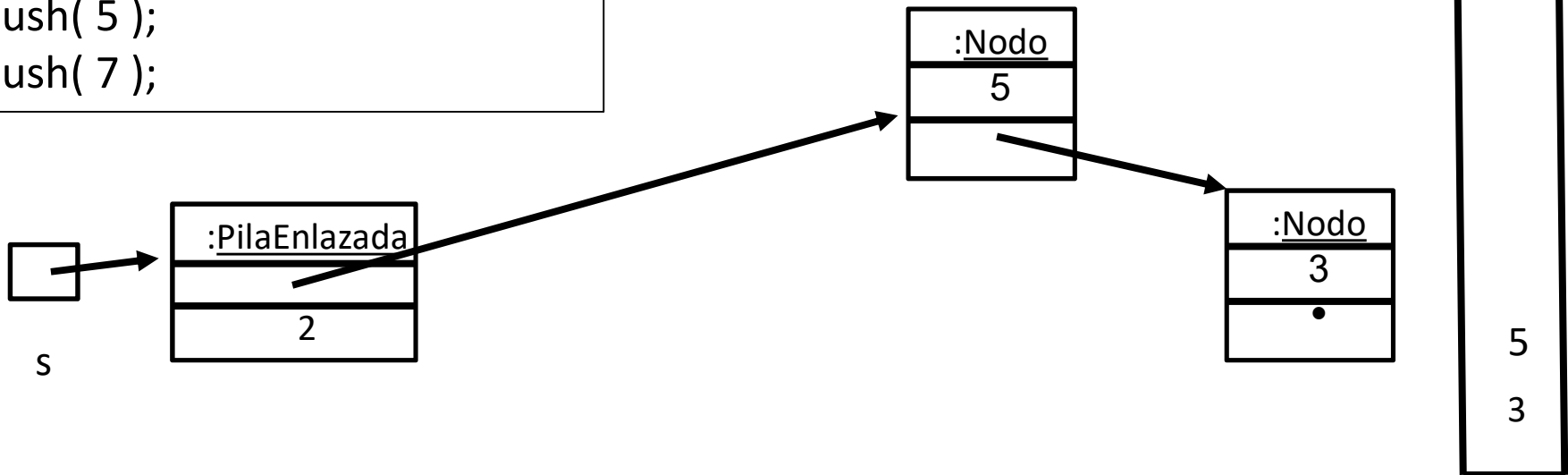
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
s.push( 7 );
```

Ahora queremos apilar un 7



PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

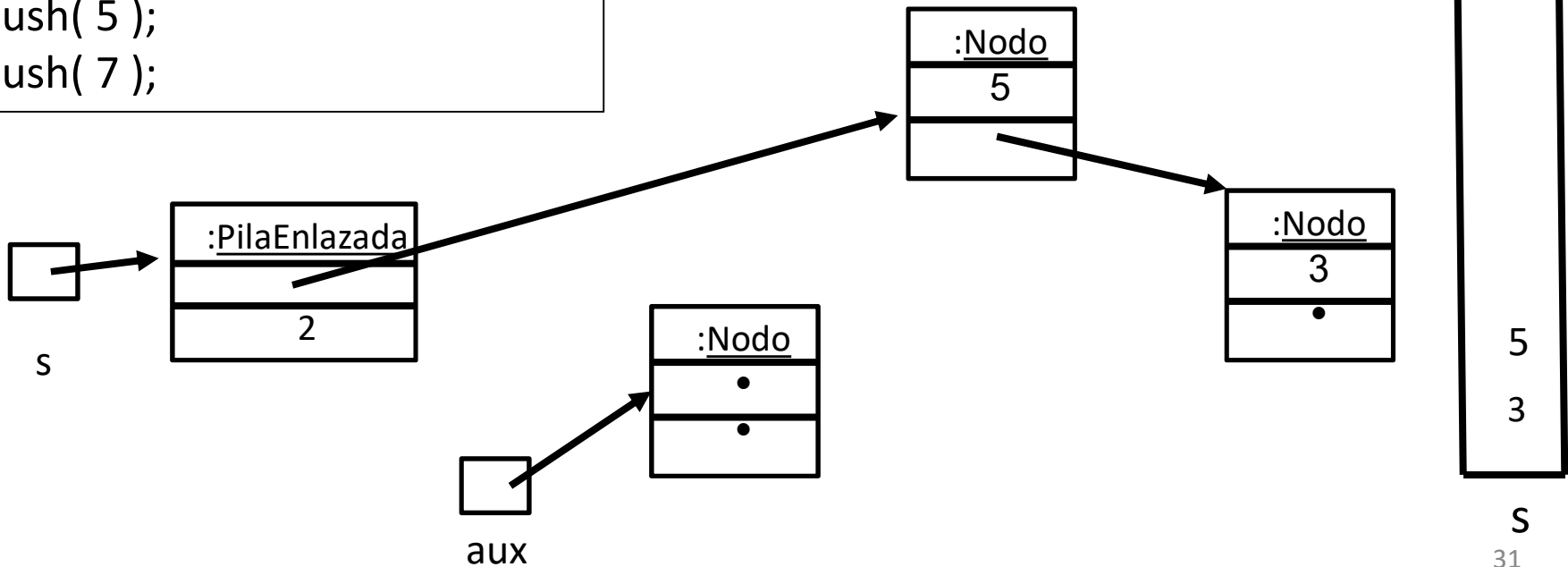
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
s.push( 7 );
```

Ejecutamos push.1: Se crea un nodo nuevo y su dirección se almacena en *aux*



PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

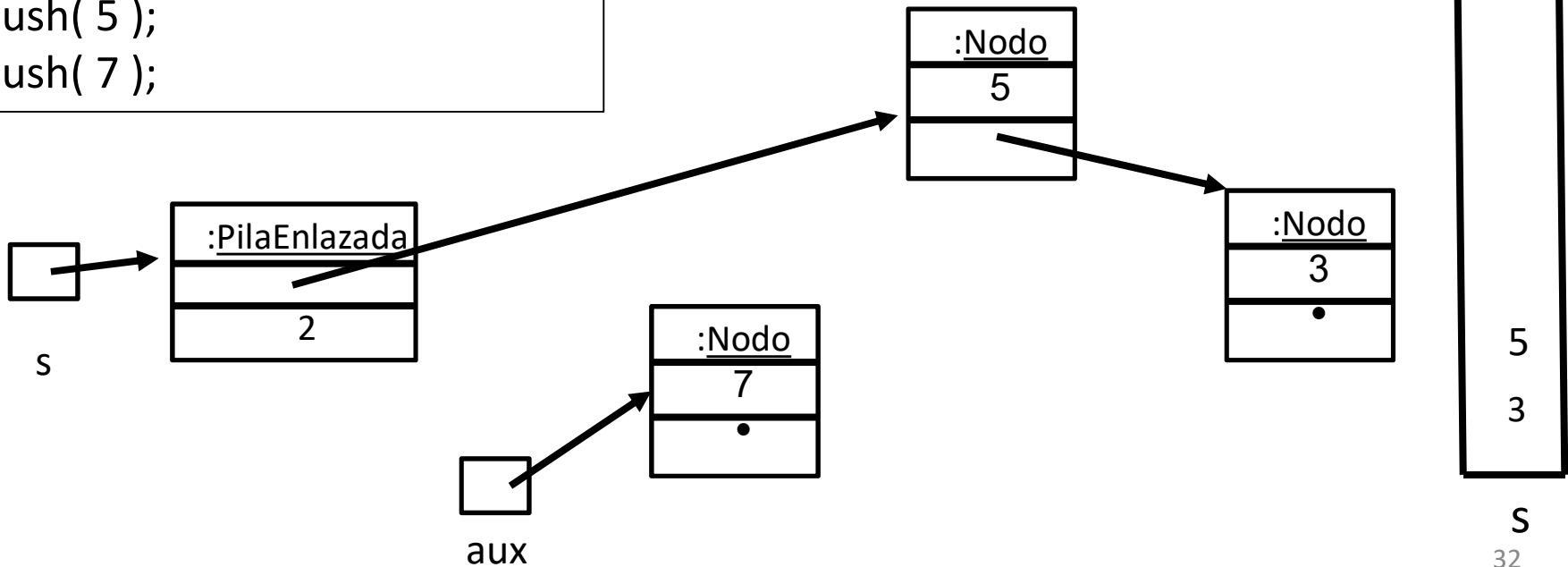
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
s.push( 7 );
```

Ejecutamos push.2: almacenamos item en el nodo



PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

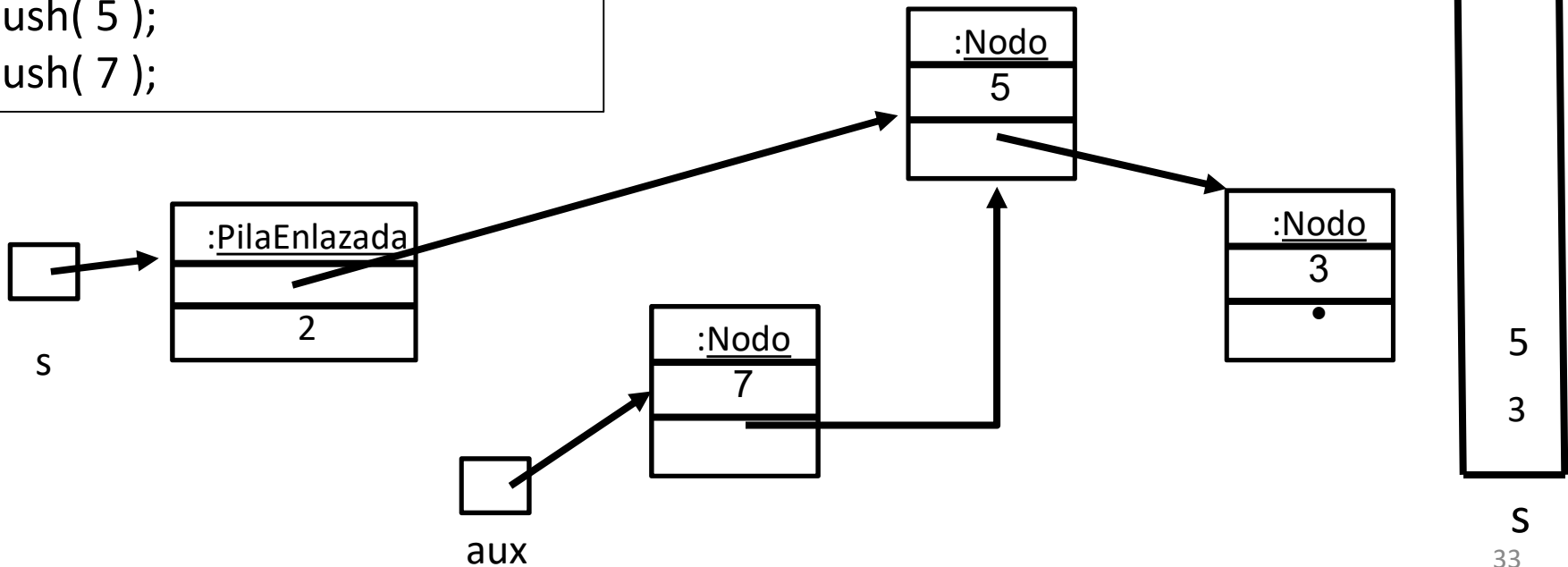
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

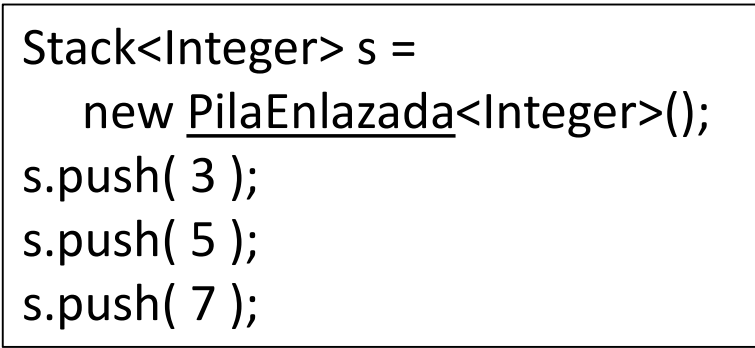
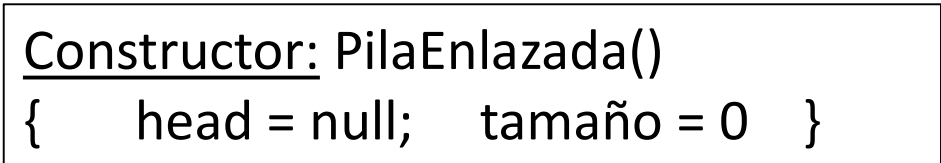
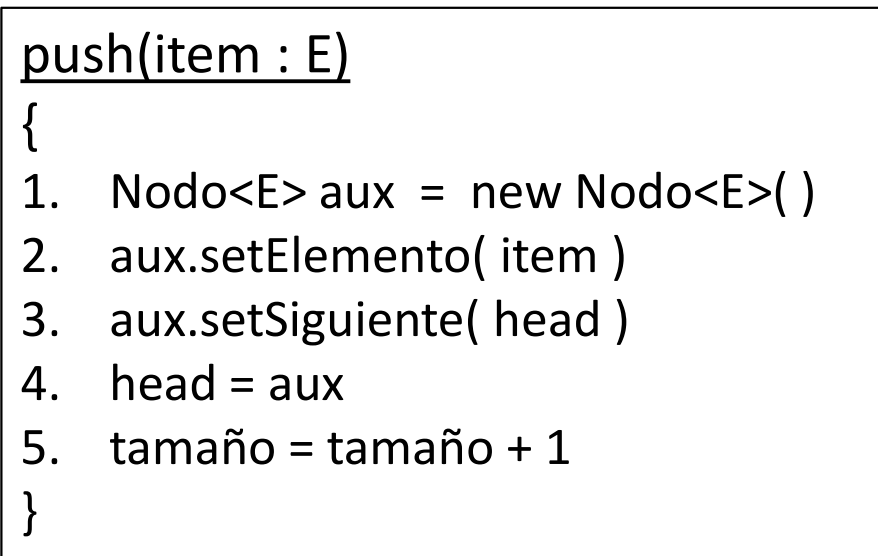
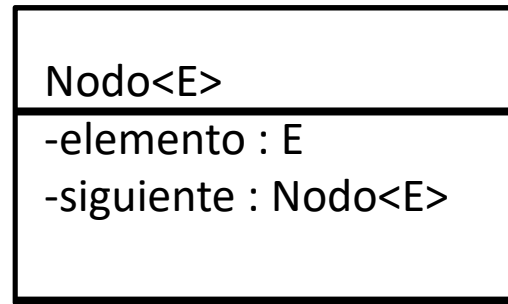
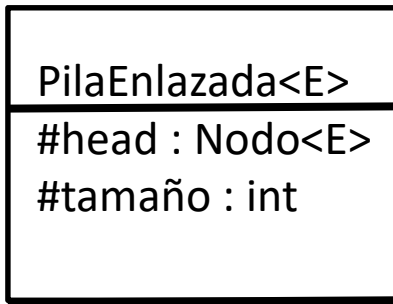
Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

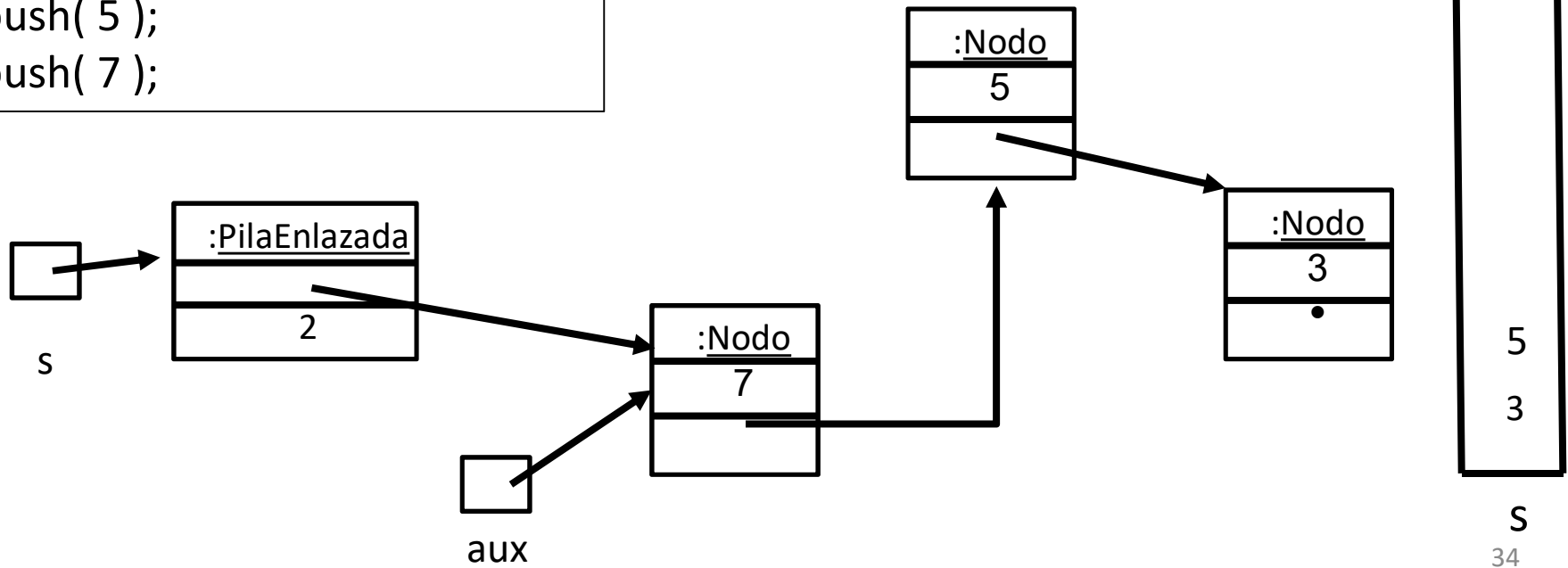
```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
s.push( 7 );
```

Ejecutamos push.3: Hacemos que el siguiente del nodo apuntado por *aux* sea *head*





Ejecutamos push.4: Hacemos que *head* apunte al nodo apuntado por *aux*



PilaEnlazada<E>
#head : Nodo<E>
#tamaño : int

Nodo<E>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

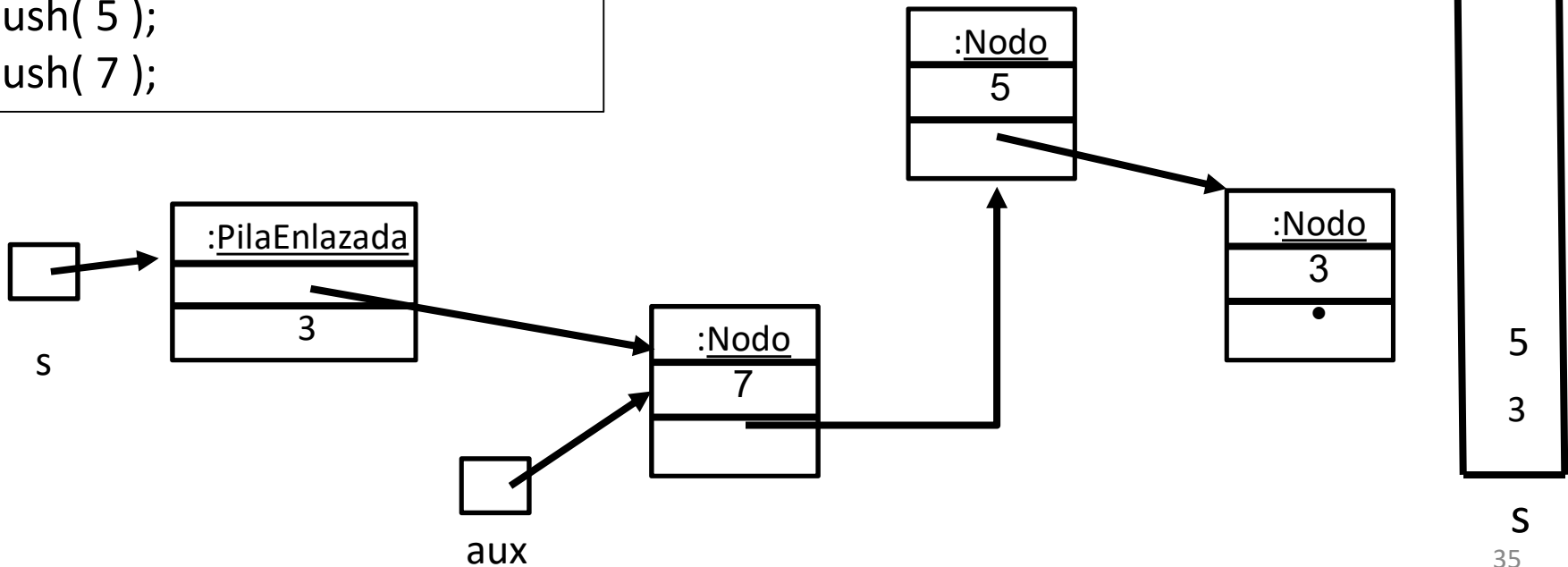
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
s.push( 7 );
```

Ejecutamos push.5: Incrementamos el tamaño de la pila a 3 elementos



<u>PilaEnlazada<E></u>
#head : Nodo<E>
#tamaño : int

<u>Nodo<E></u>
-elemento : E
-siguiente : Nodo<E>

push(item : E)

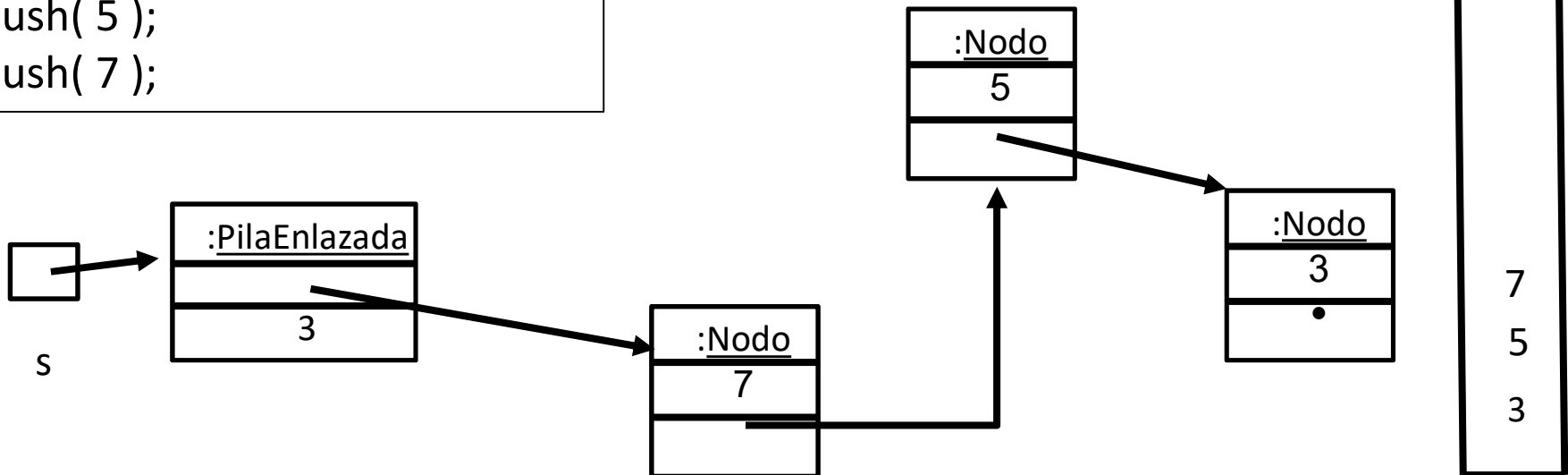
```
{
1.  Nodo<E> aux = new Nodo<E>( )
2.  aux.setElemento( item )
3.  aux.setSiguiente( head )
4.  head = aux
5.  tamaño = tamaño + 1
}
```

Constructor: PilaEnlazada()

```
{   head = null;   tamaño = 0   }
```

```
Stack<Integer> s =
    new PilaEnlazada<Integer>();
s.push( 3 );
s.push( 5 );
s.push( 7 );
```

Retornamos: Se destruye la variable local *aux* y la pila tiene un nuevo elemento



Para pensar: Variantes en la implementación de apilar

push(item : E)

```
{  
1.  Nodo<E> aux = new Nodo<E>( )  
2.  aux.setElemento( item )  
3.  aux.setSiguiente( head )  
4.  head = aux  
5.  tamaño = tamaño + 1  
}
```

(1), (2) y (3) se pueden juntar

push(item : E)

```
{  
  Nodo<E> aux = new Nodo<E>(ítem, head)  
  head = aux  
  tamaño = tamaño + 1  
}
```

(1) y (2) se
pueden juntar

push(item : E)

```
{  
  Nodo<E> aux = new Nodo<E>(item)  
  aux.setSiguiente( head )  
  head = aux  
  tamaño = tamaño + 1  
}
```

¡(1), (2), (3) y (4) se pueden juntar!

push(item : E)

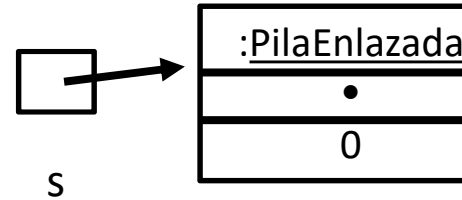
```
{  
  head = new Nodo<E>(ítem, head)  
  tamaño++  
}
```

Operación isEmpty: boolean vacia = s.isEmpty()

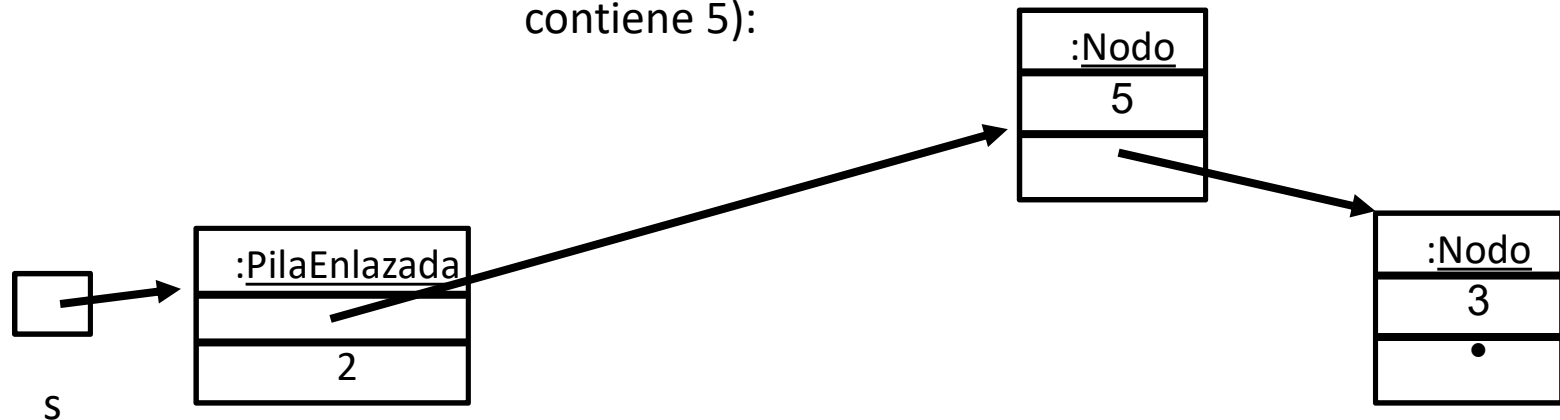
```
isEmpty() : boolean
```

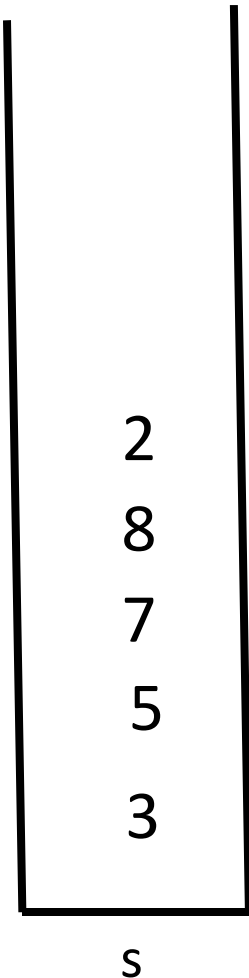
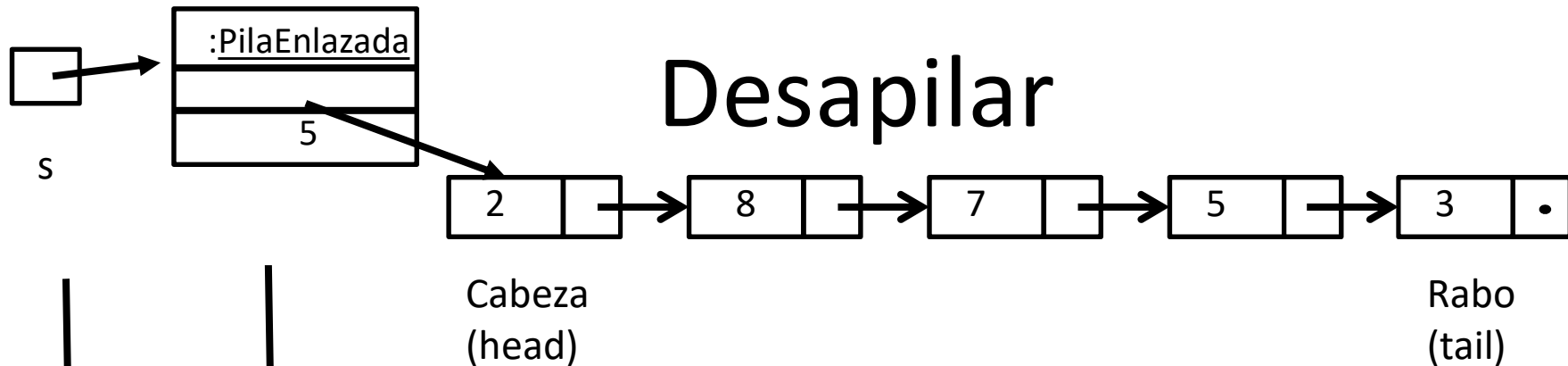
```
{  
  return head == null  
}
```

En este caso como la pila *s* está vacía el campo *head* contiene null:



En este otro caso como la pila *s* no está vacía el campo *head* apunta al nodo tope (que contiene 5):





Desapilar requiere eliminar la celda que contiene el 2 y hacer que la celda que contiene el 8 sea la nueva cabeza.

Nota: Como la celda del 2 no está referenciada por nadie, eventualmente será reclamada por el recolector de basura aunque la celda del 2 siga apuntando a la celda del 8.

Nota: Cuando la pila tiene un único elemento, el código tiene como efecto que head termine con valor null.

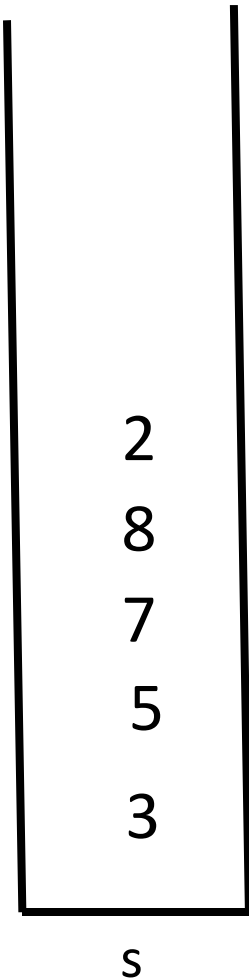
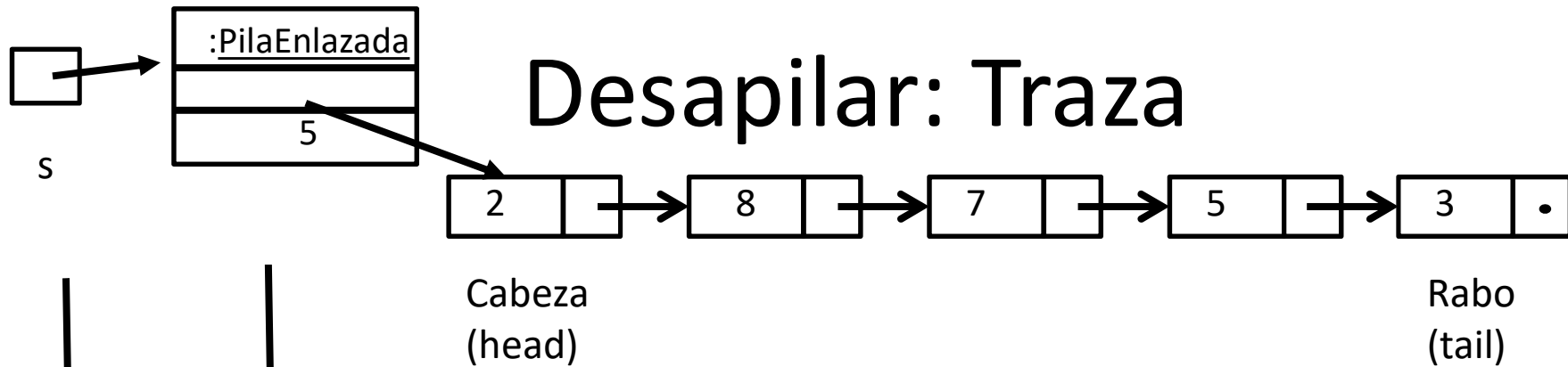
```
pop(): E {
```

```
Si isEmpty() entonces
    error( "Pila vacia" )
```

```
Sino
```

```
    aux = head.getElemento()
    head = head.getSiguiente()
    tamaño = tamaño - 1
    retornar aux
```

```
}
```



pop(): E {

Si isEmpty() entonces
error("Pila vacia")

Sino

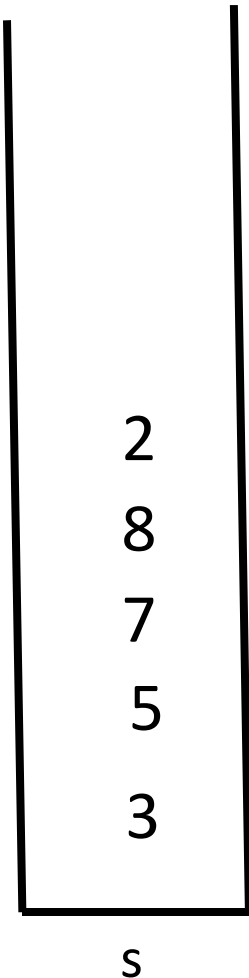
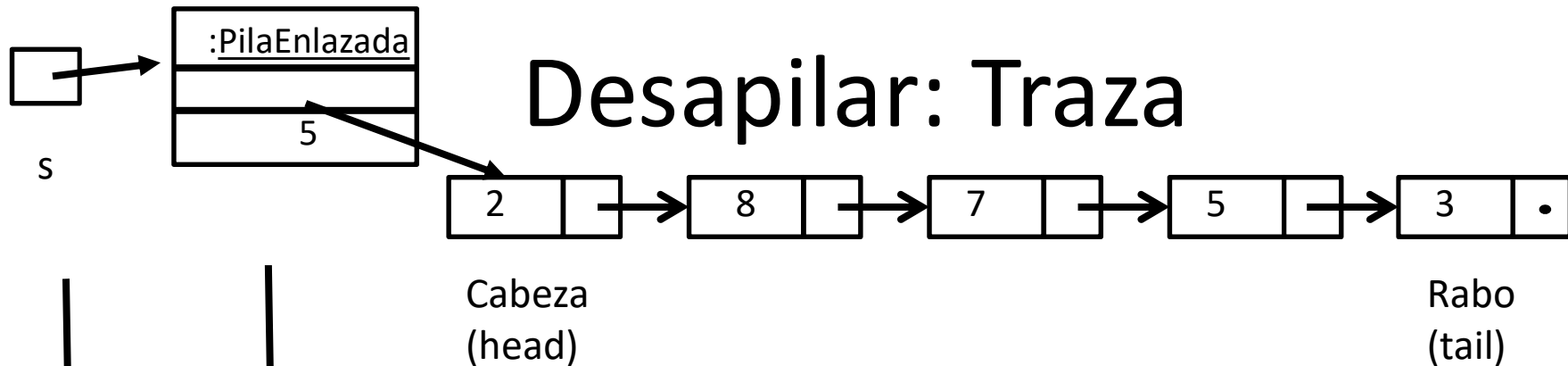
1. aux = head.getElemento()

2. head = head.getSiguiente()

3. tamaño = tamaño - 1

4. retornar aux

}



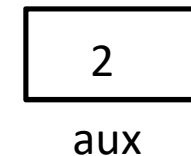
pop(): E {

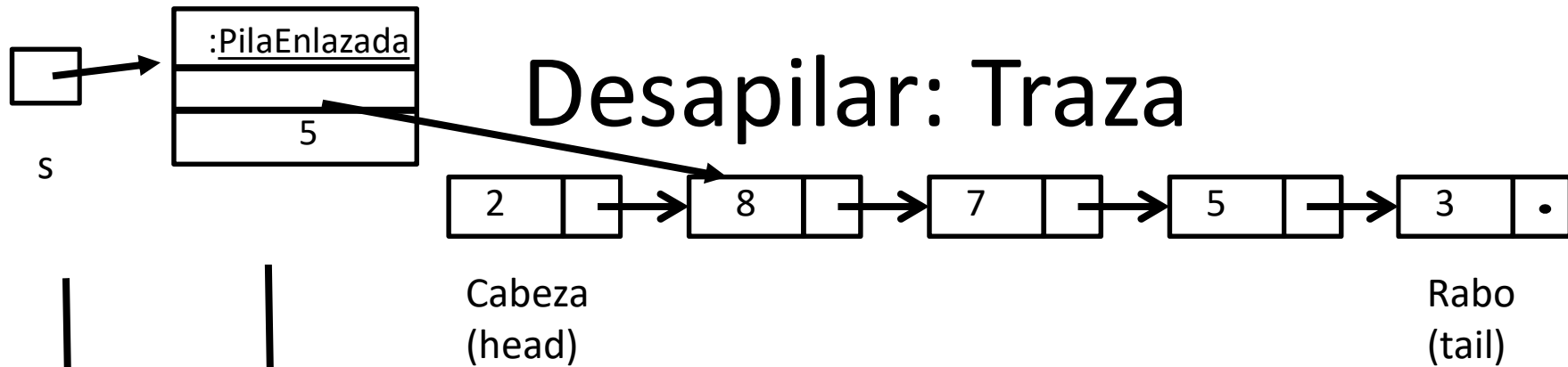
Si isEmpty() entonces
error("Pila vacia")

Sino

1. aux = head.getElemento()
 2. head = head.getSiguiente()
 3. tamaño = tamaño - 1
 4. retornar aux
- }

Como la pila no está vacía, ejecutamos (1.) y almacenamos el elemento de head en aux de tipo E (que en este caso es un Integer)





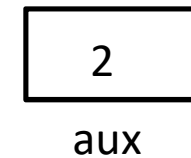
pop(): E {

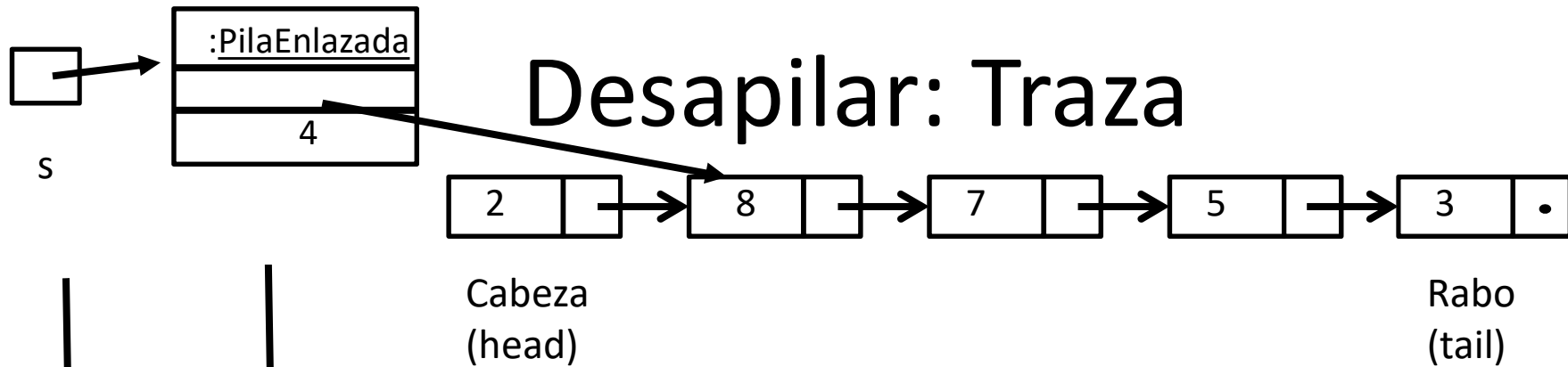
Si isEmpty() entonces
error("Pila vacia")

Sino

1. aux = head.getElemento()
 2. head = head.getSiguiente()
 3. tamaño = tamaño - 1
 4. retornar aux
- }

Al ejecutar (2) hacemos un bypass de head al segundo elemento





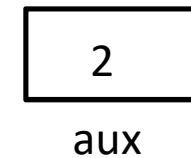
pop(): E {

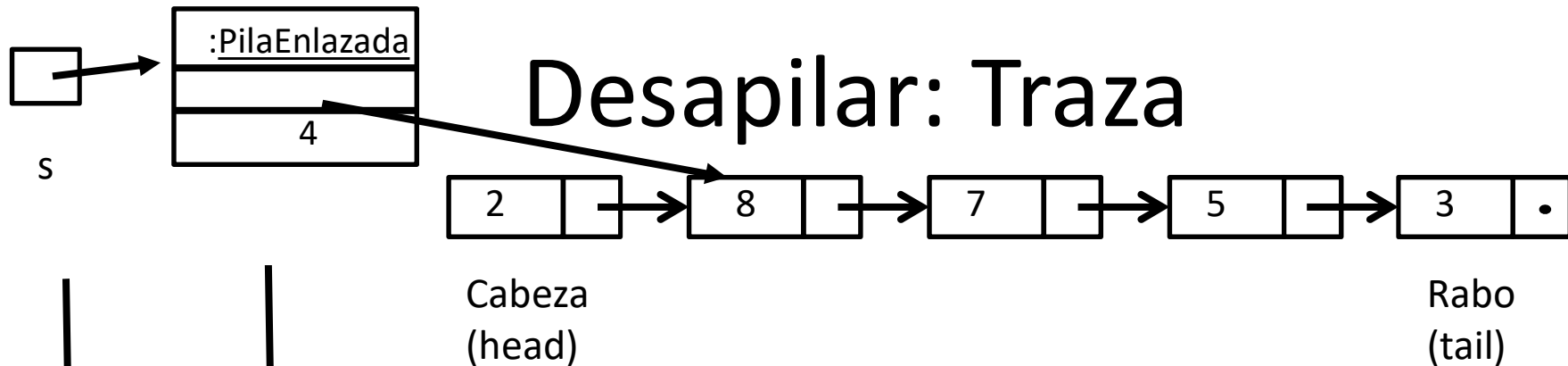
Si isEmpty() entonces
error("Pila vacia")

Sino

1. `aux = head.getElemento()`
 2. `head = head.getSiguiente()`
 3. `tamaño = tamaño - 1`
 4. retornar `aux`
- }

Al ejecutar (3)
decrementamos el tamaño
a 4.





pop(): E {

Si isEmpty() entonces
error("Pila vacia")

Sino

1. aux = head.getElemento()
 2. head = head.getSiguiente()
 3. tamaño = tamaño - 1
 4. retornar aux
- }

Al ejecutar (4) retornamos el 2 que está en aux y aux se destruye porque es una variable local. El nodo que tiene el 2 queda en el heap y como nadie lo referencia, en algún momento será reclamado por el recolector de basura. La pila ahora tiene como tope al 8.

En Java el error se señala con una excepción

Debemos crear una excepción de programador llamada EmptyStackException que sea subclase de RuntimeException.

```
public E pop() {  
    if isEmpty() {  
        throw new EmptyStackException( "PilaEnlazada:pop:: Pila vacia" );  
    } else {  
        E aux = head.getElemento();  
        head = head.getSiguiente();  
        tamaño--;  
        return aux;  
    }  
}
```

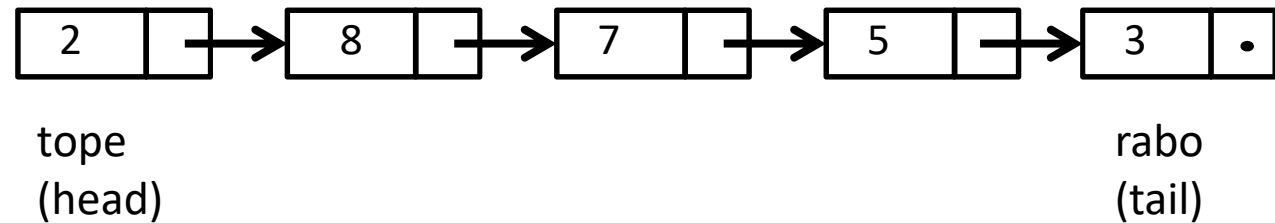
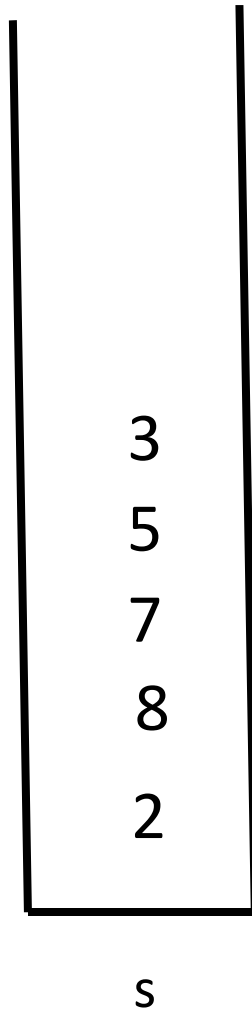
Refactorización con guarded clauses

```
public E pop() {  
    if isEmpty() {  
        throw new EmptyStackException( "PilaEnlazada:pop:: Pila vacia" );  
    } else {  
        E aux = head.getElemento();  
        head = head.getSiguiente();  
        tamaño--;  
        return aux;  
    }  
}
```

Como throw termina la ejecución del método, se reescribe como:

```
public E pop() {  
    if isEmpty() throw new EmptyStackException( "PilaEnlazada:pop:: Pila vacia" );  
    E aux = head.getElemento();  
    head = head.getSiguiente();  
    tamaño--;  
    return aux;  
}
```

Comentarios sobre estrategia incorrecta de apilado

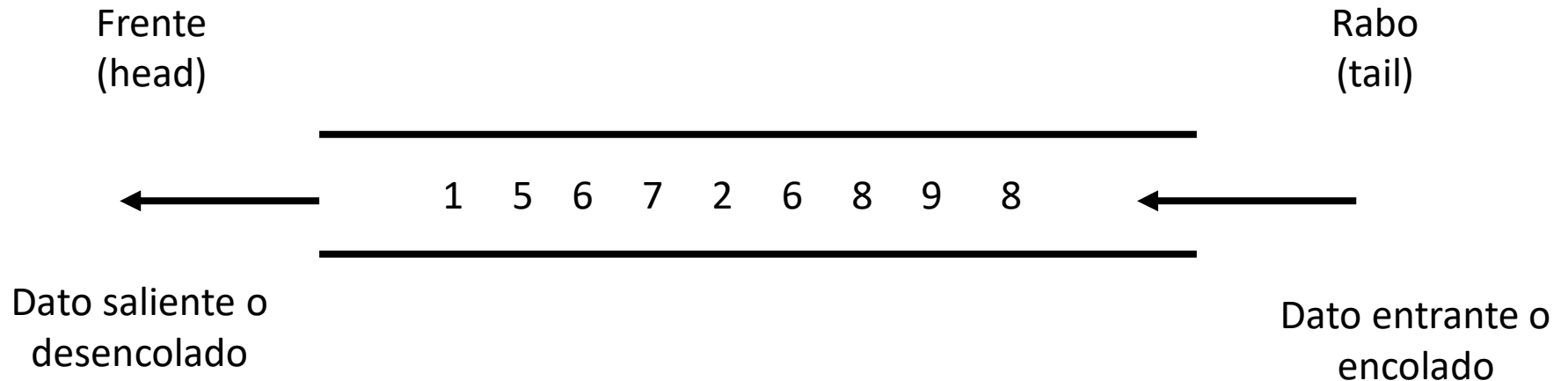


Pregunta: ¿Por qué conviene insertar en la cabeza de la lista y no en el rabo?

Respuesta: Si insertáramos en el rabo, la lista de nodos representaría la pila de la izquierda.
En este caso, hacer una inserción requiere recorrer toda la lista.

Conclusión: Insertar en el rabo sería ineficiente y NO LO VAMOS A HACER.

TDA Cola

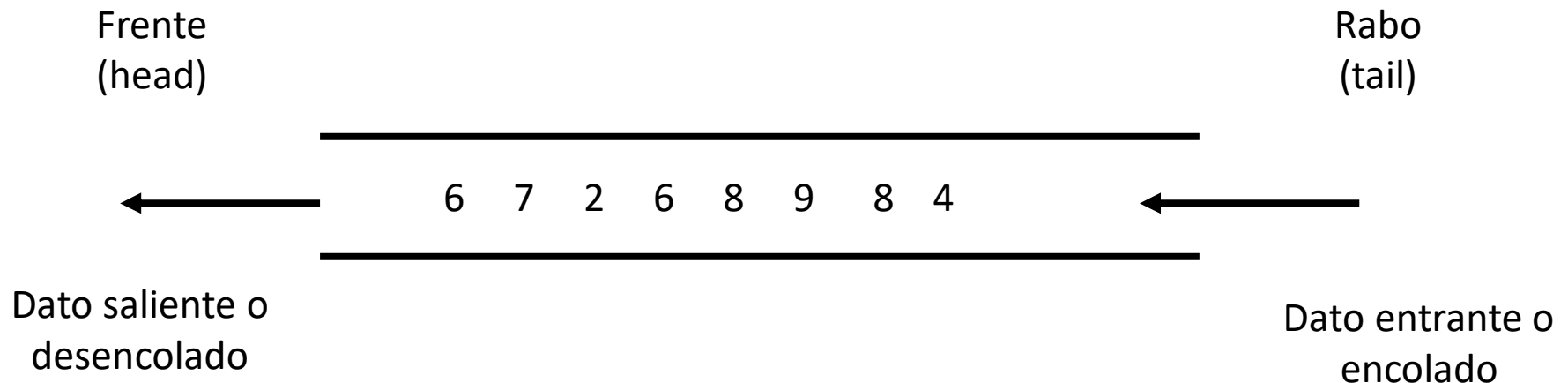


Una *cola* (queue) es una colección con modelo de secuencia cuyos datos ingresan por un extremo llamado *rabo* (tail) y salen por otro extremo llamado *frente* (head).

Esta política de actualización se llama FIFO (por *first-in first-out*) o también FCFS (*first-come first-served*).


```
public interface Queue<E> {  
    // Inserta el elemento e al final de la cola  
    public void enqueue(E item);  
  
    // Elimina el elemento del frente de la cola y lo retorna.  
    // Si la cola está vacía se produce un error.  
    public E dequeue();  
  
    // Retorna el elemento del frente de la cola.  
    // Si la cola está vacía se produce un error.  
    public E front();  
  
    // Retorna verdadero si la cola no tiene elementos  
    // y falso en caso contrario  
    public boolean isEmpty();  
  
    // Retorna la cantidad de elementos de la cola.  
    public int size();  
}
```

TDA Cola

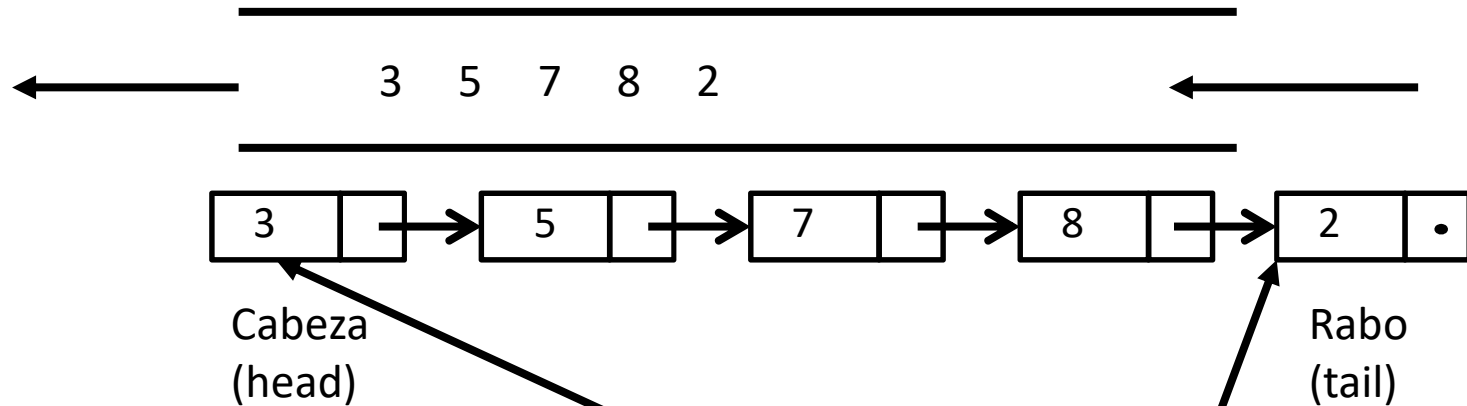


```
Queue<Integer> q = new ColaEnlazada<Integer>();
q.enqueue(1);
q.enqueue(5);
q.enqueue(6);
q.enqueue(7);
....
q.enqueue(8);
int elemento = q.dequeue(); // elemento vale 1 y se borra de la cola
q.dequeue();               // borro el 2 de la cola y no lo almaceno
q.enqueue(4);              // Puedo encolar cuando quiera
```

Implementaciones de colas

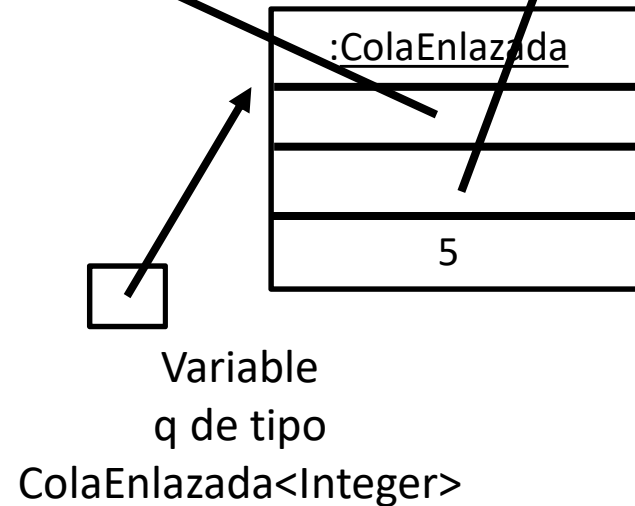
- 1) Con un arreglo circular:
 - 1) Requiere el tamaño inicial
 - 2) Requiere aritmética de módulo para funcionar eficientemente (para no para no hacer corrimientos al eliminar)
 - 3) Presenta complicaciones para agrandar el arreglo cuando se agota la capacidad
- 2) Con una estructura enlazada:
 - 1) No requiere tamaño inicial
 - 2) Requiere mantener la información de los dos extremos para implementar en forma eficiente
 - 3) La estructura es no acotada
- 3) En términos de una lista (lo dejamos pendiente hasta dar el TDA Lista)

Cola: Implementación con nodos enlazados



```
public class ColaEnlazada<E>
    implements Queue<E> {
    protected Nodo<E> head, tail;
    protected int tamaño;

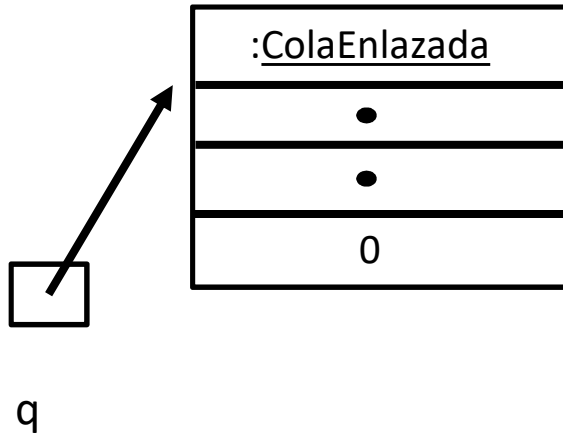
    ....
}
```



Nota: No va a haber límite en la cantidad de elementos que se pueden encolar.

Cola enlazada: Constructor

Queue<Integer> q = new ColaEnlazada<Integer>();



```
public class ColaEnlazada<E>
    implements Queue<E> {
    protected Nodo<E> head, tail;
    protected int tamaño;

    ....
}
```

```
public ColaEnlazada() {
    head = null;
    tail = null;
    tamaño = 0;
}
```

Hacia la implementación de encolar y desencolar

Heurísticas a la hora de plantear implementación:

Plantear un caso cuando la estructura está vacía.

Plantear otro caso cuando la estructura tiene un único nodo.

Plantear otro caso cuando la estructura tiene 2 nodos o más.

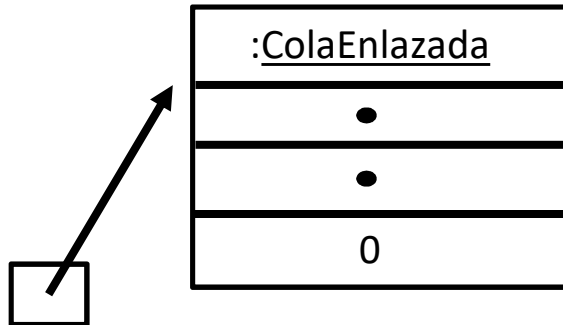
Como tenemos 3 alternativas, la estructura de control a elegir será un if-then-else anidado.

Luego podemos ver que hay casos que son iguales y se pueden refactorizar.

Luego podemos ver que hay sentencias comunes al principio o al final de los if-else y éstas se pueden factorizar arriba del if-else o debajo según corresponda.

En el caso de esta teoría mostramos la implementación de libro que ya tiene en cuenta estas cuestiones.

Cola: encolar en la cola vacía



q

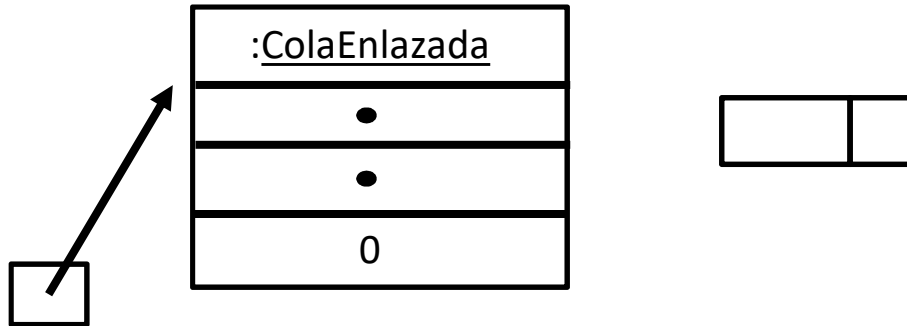
Quiero ejecutar:

q.enqueue(3);

Veamos la traza:

```
public void enqueue( E item ) {  
1.  Nodo<E> nodo = new Nodo<E>();  
2.  nodo.setElemento( item );  
3.  nodo.setSiguiente( null );  
4.  if ( head == null )  
5.      head = nodo;  
6.  else  
7.      tail.setSiguiente( nodo );  
8.  tail = nodo;  
9.  tamaño++;  
}
```

Cola: encolar en la cola vacía



`q`

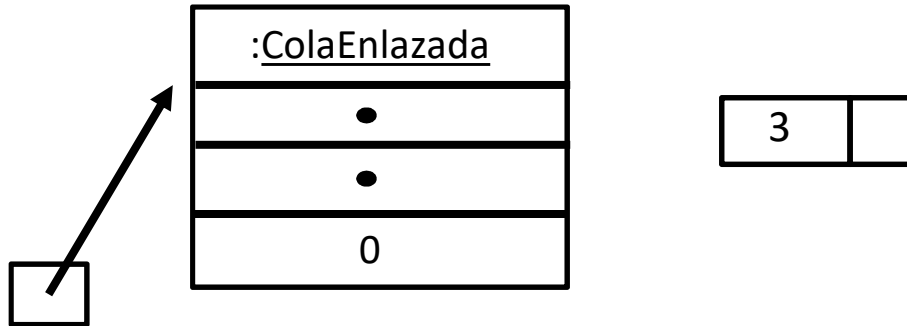
Quiero ejecutar:

`q.enqueue(3);`

```
public void enqueue( E item ) {  
1.  Nodo<E> nodo = new Nodo<E>();
```

```
}
```


Cola: encolar en la cola vacía



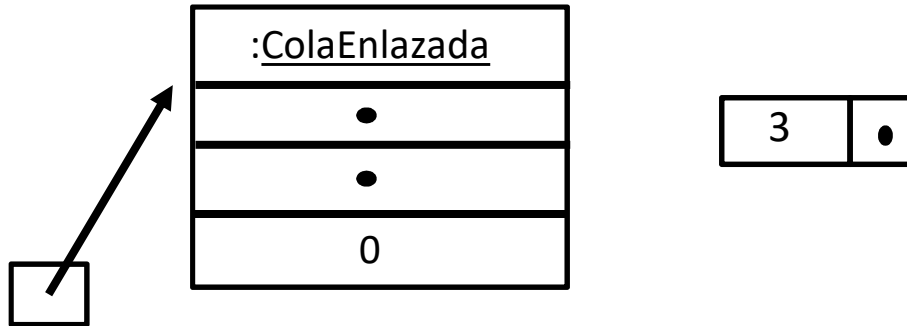
q

Quiero ejecutar:

q.enqueue(3);

```
public void enqueue( E item ) {  
    1.  Nodo<E> nodo = new Nodo<E>();  
    2.  nodo.setElemento( item );  
  
}
```

Cola: encolar en la cola vacía



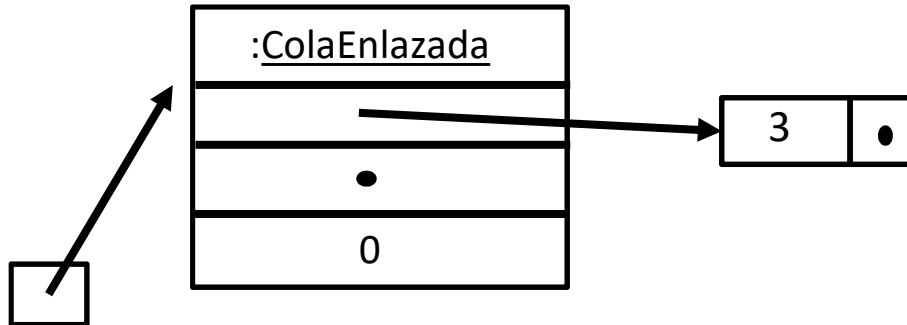
`q`

Quiero ejecutar:

`q.enqueue(3);`

```
public void enqueue( E item ) {  
    1.  Nodo<E> nodo = new Nodo<E>();  
    2.  nodo.setElemento( item );  
    3.  nodo.setSiguiente( null );  
  
}
```

Cola: encolar en la cola vacía



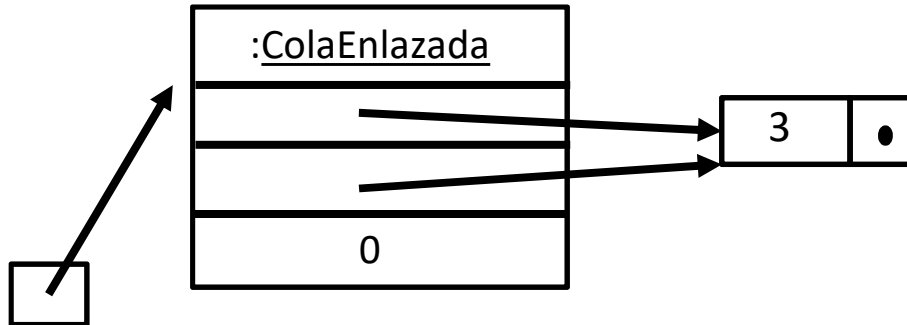
`q`

Quiero ejecutar:

`q.enqueue(3);`

```
public void enqueue( E item ) {  
    1.  Nodo<E> nodo = new Nodo<E>();  
    2.  nodo.setElemento( item );  
    3.  nodo.setSiguiente( null );  
    4.  if ( head == null )  
    5.      head = nodo;  
    6.  else  
    7.      tail.setSiguiente( nodo );  
}
```

Cola: encolar en la cola vacía



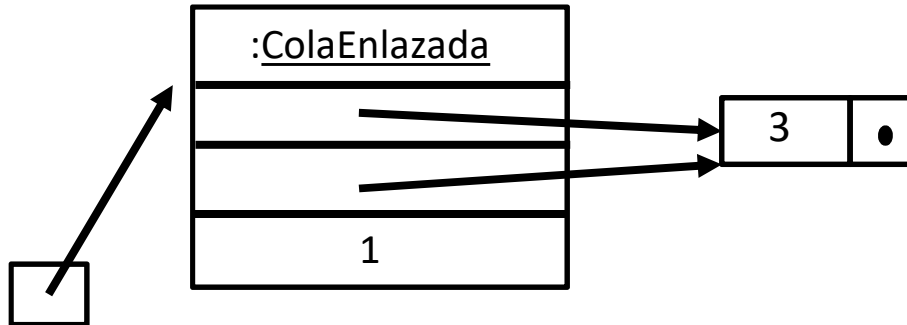
q

Quiero ejecutar:

q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiente( nodo );
8.  tail = nodo;
}
```

Cola: encolar en la cola vacía



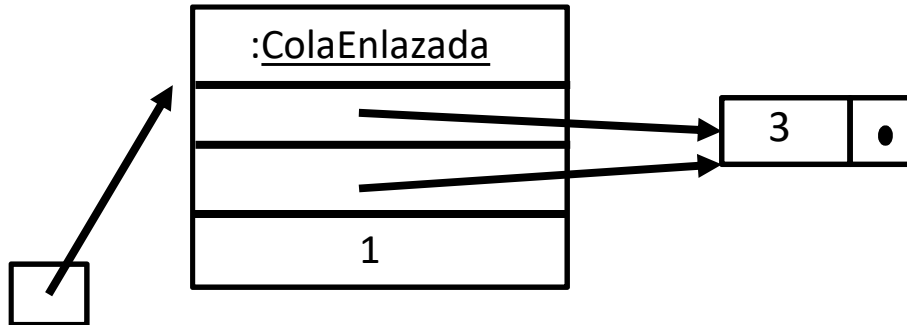
q

Quiero ejecutar:

q.enqueue(3);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiente( nodo );
8.  tail = nodo;
9.  tamaño++;
}
```

Cola: encolar en la cola no vacía



`q`

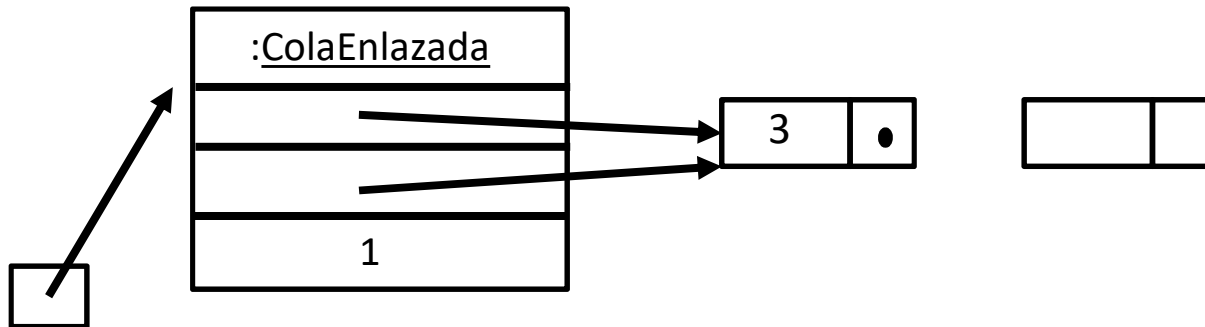
Quiero ejecutar:

`q.enqueue(5);`

Veamos la traza:

```
public void enqueue( E item ) {  
1.  Nodo<E> nodo = new Nodo<E>();  
2.  nodo.setElemento( item );  
3.  nodo.setSiguiente( null );  
4.  if ( head == null )  
5.      head = nodo;  
6.  else  
7.      tail.setSiguiente( nodo );  
8.  tail = nodo;  
9.  tamaño++;  
}
```

Cola: encolar en la cola no vacía



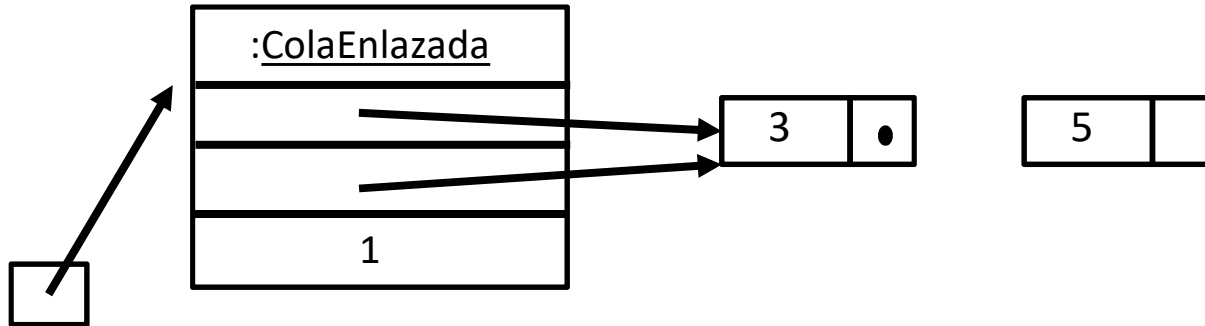
q

Quiero ejecutar:

```
q.enqueue(5);
```

```
public void enqueue( E item ) {  
    1.  Nodo<E> nodo = new Nodo<E>();
```

Cola: encolar en la cola no vacía



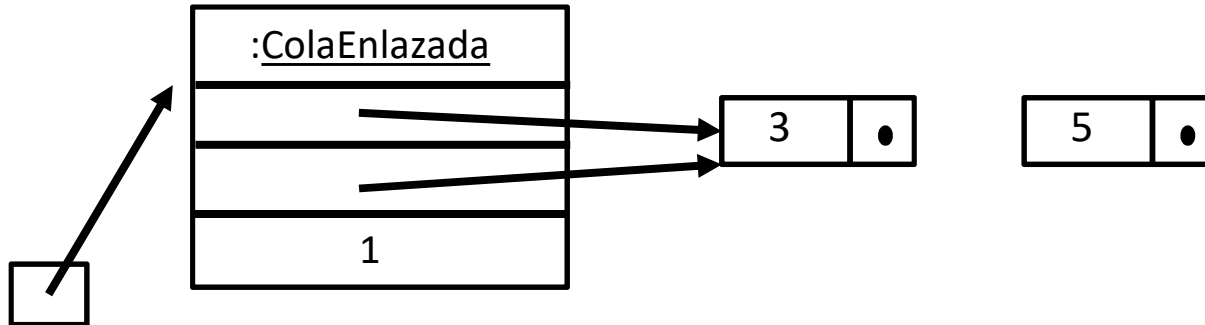
`q`

Quiero ejecutar:

`q.enqueue(5);`

```
public void enqueue( E item ) {  
    1. Nodo<E> nodo = new Nodo<E>();  
    2. nodo.setElemento( item );  
  
}
```


Cola: encolar en la cola no vacía



q

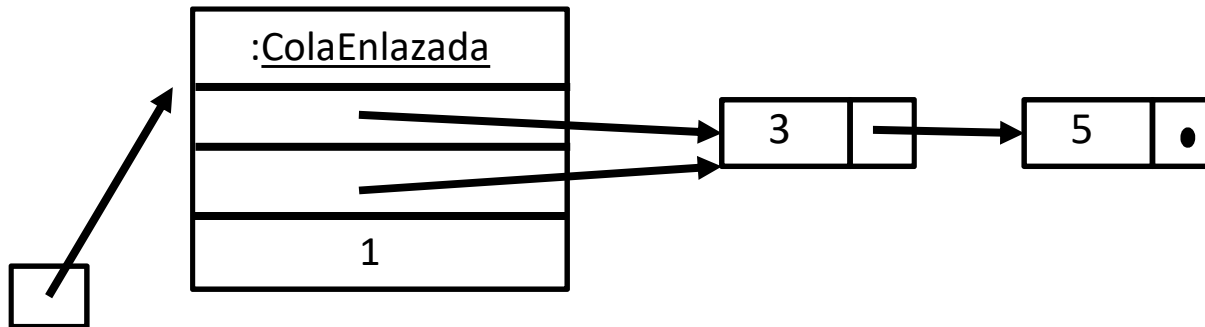
Quiero ejecutar:

q.enqueue(5);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );

}
```

Cola: encolar en la cola no vacía



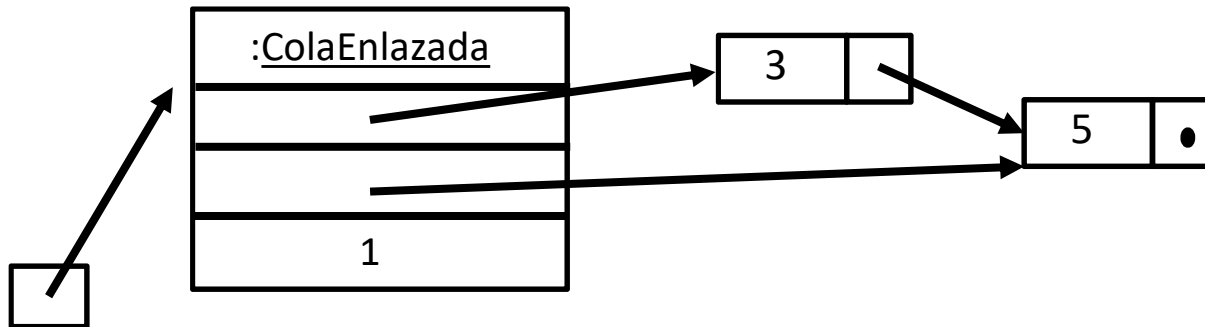
q

Quiero ejecutar:

q.enqueue(5);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiente( nodo );
}
```

Cola: encolar en la cola no vacía



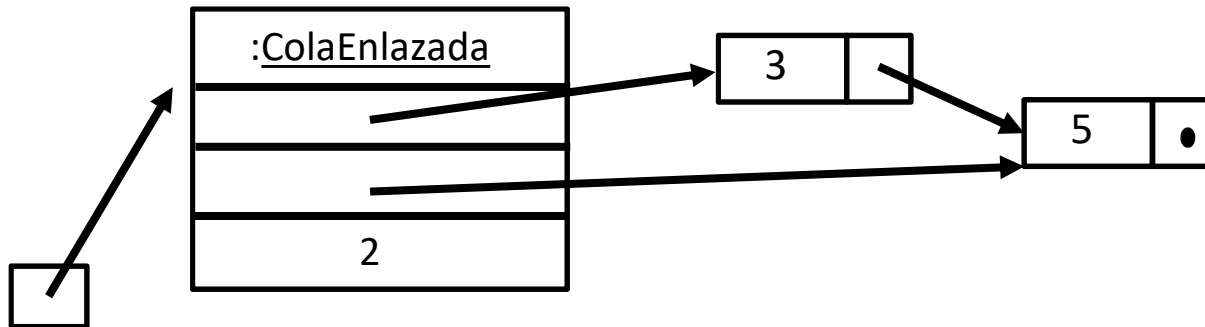
q

Quiero ejecutar:

q.enqueue(5);

```
public void enqueue( E item ) {
1.  Nodo<E> nodo = new Nodo<E>();
2.  nodo.setElemento( item );
3.  nodo.setSiguiente( null );
4.  if ( head == null )
5.      head = nodo;
6.  else
7.      tail.setSiguiente( nodo );
8.  tail = nodo;
}
```

Cola: encolar en la cola no vacía



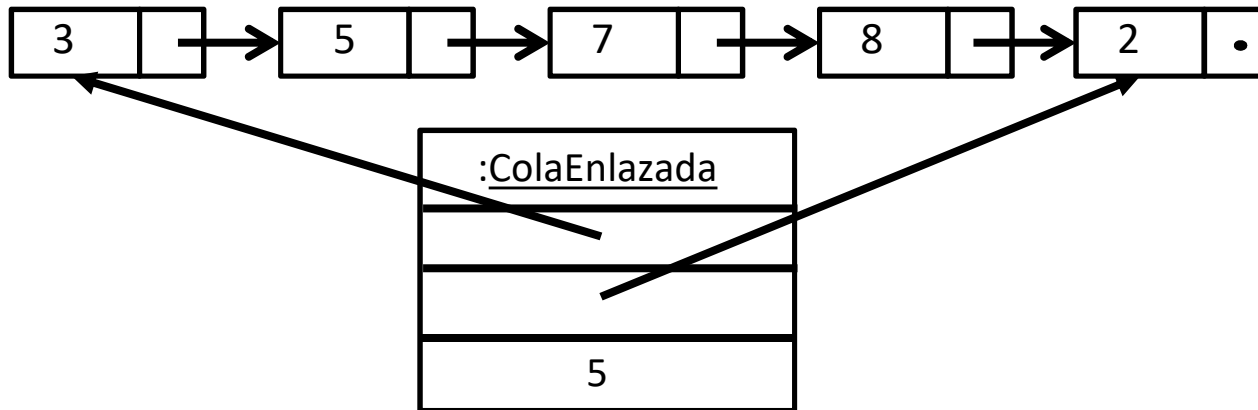
`q`

Quiero ejecutar:

`q.enqueue(5);`

```
public void enqueue( E item ) {  
1.  Nodo<E> nodo = new Nodo<E>();  
2.  nodo.setElemento( item );  
3.  nodo.setSiguiente( null );  
4.  if ( head == null )  
5.      head = nodo;  
6.  else  
7.      tail.setSiguiente( nodo );  
8.  tail = nodo;  
9.  tamaño++;  
}
```

Cola: encolar en cola no vacía



Quiero ejecutar

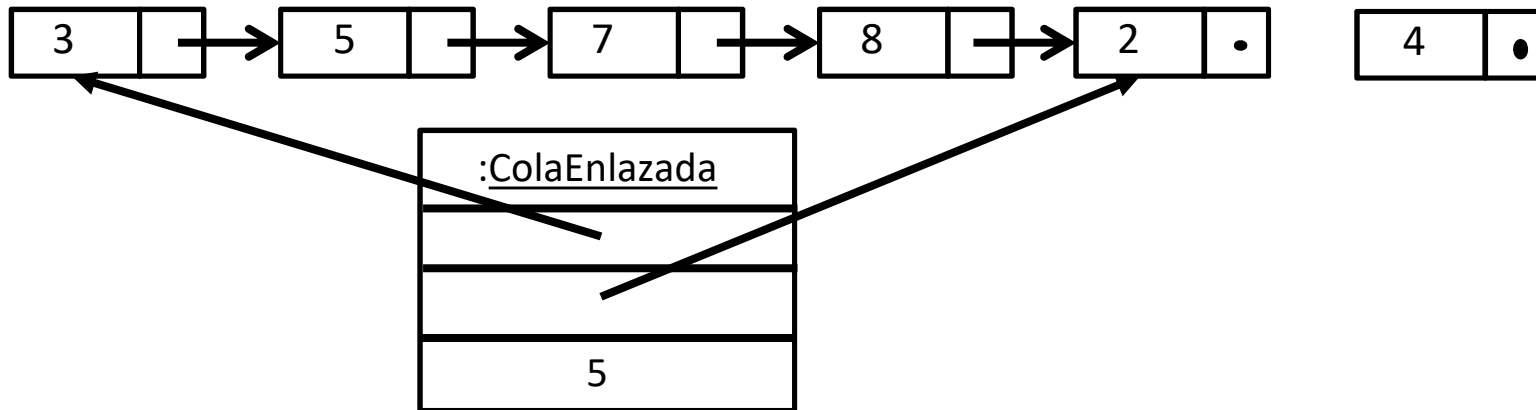
`q.enqueue(4);`

Veamos la traza

```
public void enqueue( E item ) {
```

```
}
```

Cola: encolar en cola no vacía



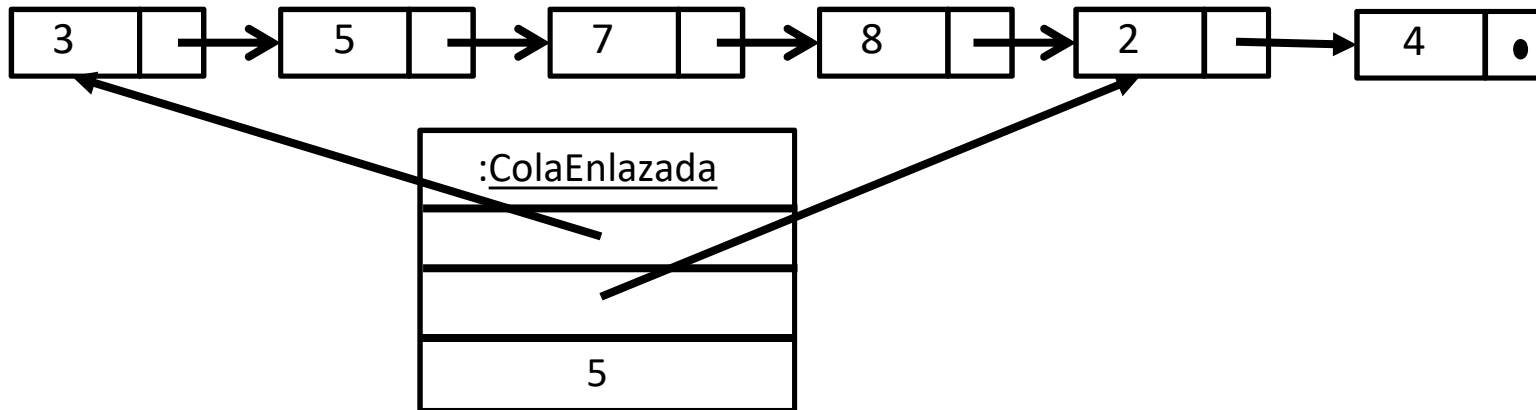
Quiero ejecutar

`q.enqueue(4);`

Veamos la traza

```
public void enqueue( E item ) {  
    1.  Nodo<E> nodo = new Nodo<E>();  
    2.  nodo.setElemento( item );  
    3.  nodo.setSiguiente( null );  
  
}
```

Cola: encolar en cola no vacía



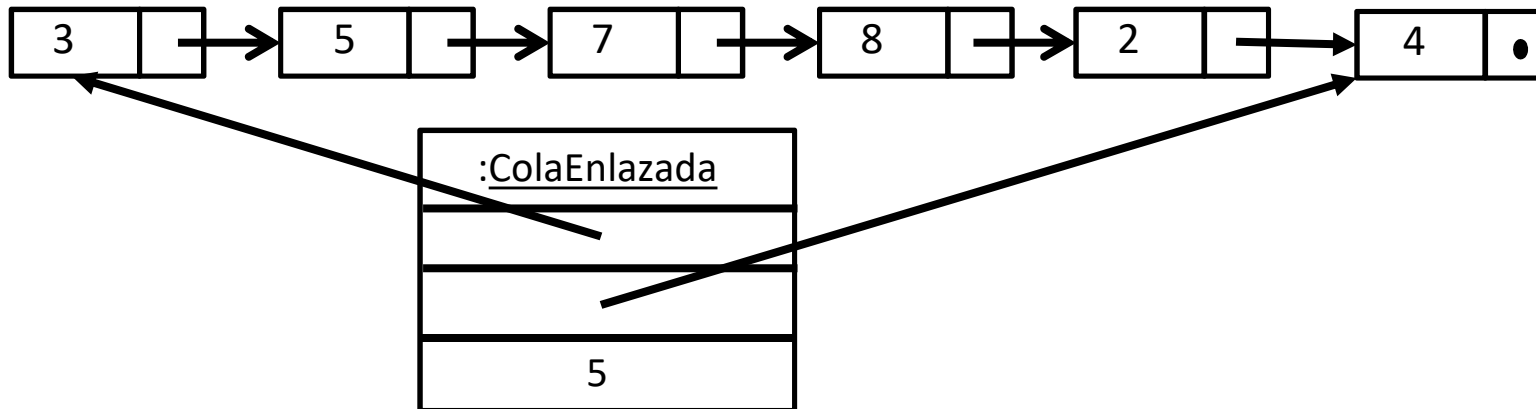
Quiero ejecutar

`q.enqueue(4);`

Veamos la traza

```
public void enqueue( E item ) {  
    1.  Nodo<E> nodo = new Nodo<E>();  
    2.  nodo.setElemento( item );  
    3.  nodo.setSiguiente( null );  
    4.  if ( head == null )  
    5.      head = nodo;  
    6.  else  
    7.      tail.setSiguiente( nodo );  
}
```

Cola: encolar en cola no vacía



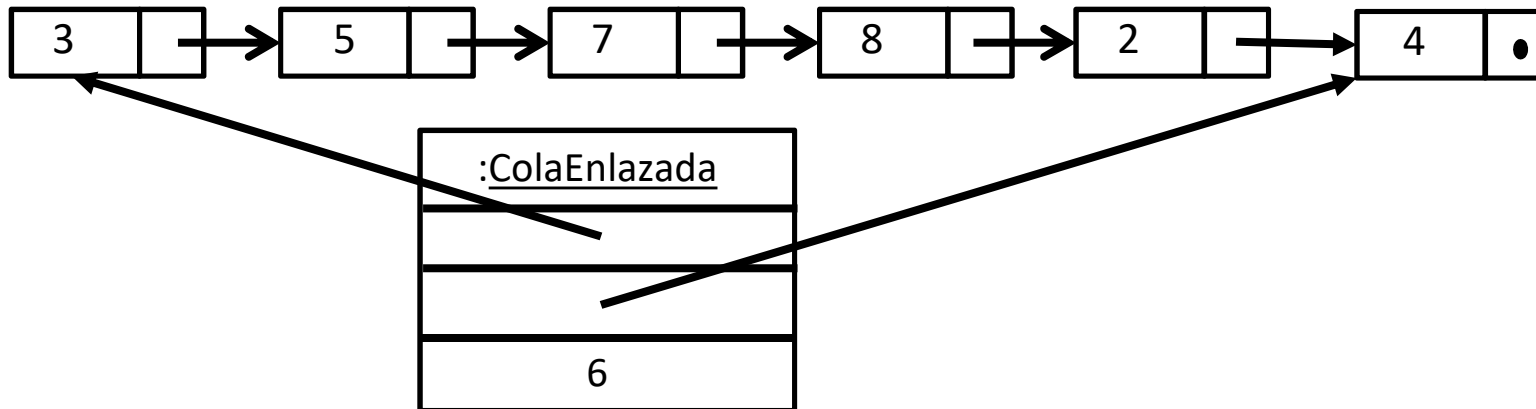
Quiero ejecutar

`q.enqueue(4);`

Veamos la traza

```
public void enqueue( E item ) {  
    1.  Nodo<E> nodo = new Nodo<E>();  
    2.  nodo.setElemento( item );  
    3.  nodo.setSiguiente( null );  
    4.  if ( head == null )  
    5.      head = nodo;  
    6.  else  
    7.      tail.setSiguiente( nodo );  
    8.  tail = nodo;  
}
```


Cola: encolar en cola no vacía



Quiero ejecutar

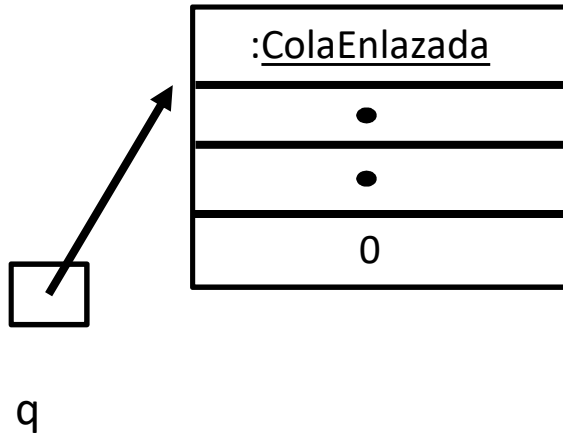
`q.enqueue(4);`

Veamos la traza

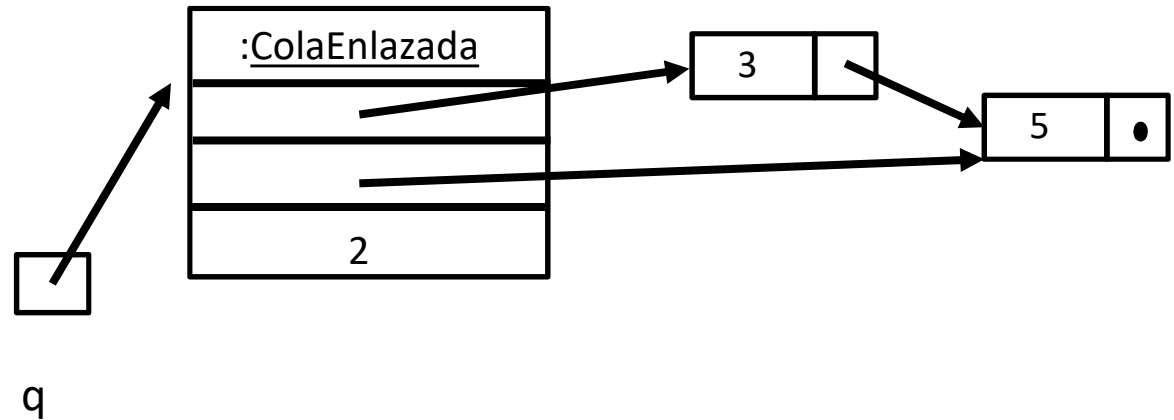
```
public void enqueue( E item ) {  
1.  Nodo<E> nodo = new Nodo<E>();  
2.  nodo.setElemento( item );  
3.  nodo.setSiguiente( null );  
4.  if ( head == null )  
5.      head = nodo;  
6.  else  
7.      tail.setSiguiente( nodo );  
8.  tail = nodo;  
9.  tamaño++;  
}
```

Implementación de isEmpty()

Caso cuando la cola está vacía:



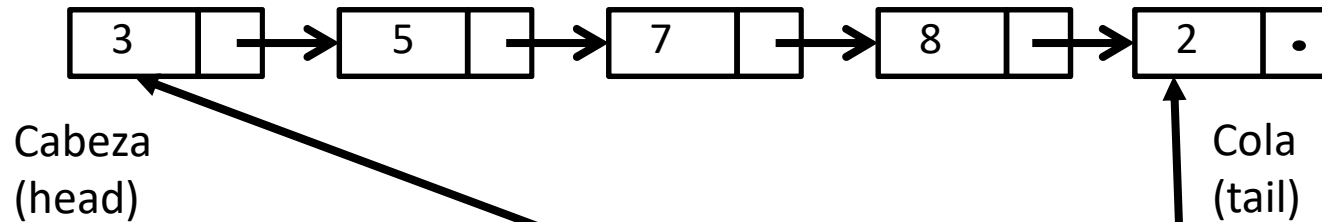
Caso cuando la cola tiene elementos:



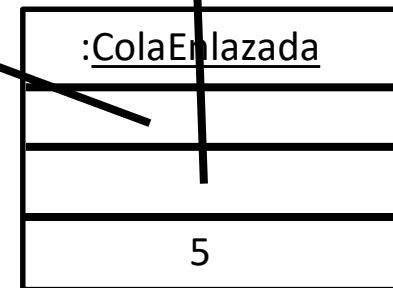
Entonces isEmpty() devuelve un booleano que se puede obtener de una de 2 formas:

1. Testear si tamaño es 0.
2. Testear si head es null.

Cola: desencolar



```
public E dequeue() {  
  1. if( isEmpty() )  
  2.   throw new EmptyQueueException( "cola vacía" );  
  
  3. E tmp = head.getElemento();  
  4. head = head.getNext();  
  5. tamaño --;  
  6. if( head == null ) tail = null;  
  7. return tmp;  
}
```

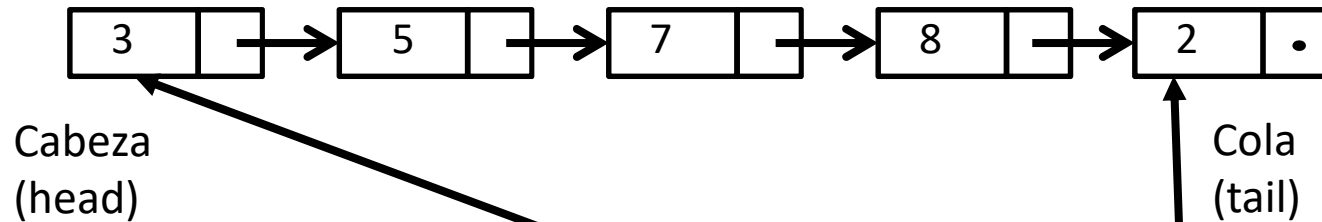


Quiero ejecutar

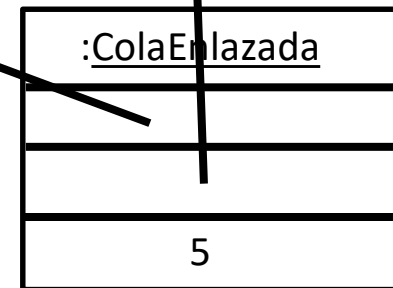
`q.dequeue();`

Veamos la traza.

Cola: desencolar



```
public E dequeue() {  
  1. if( isEmpty() )  
  2.   throw new EmptyQueueException( "cola vacía" );  
  
}
```

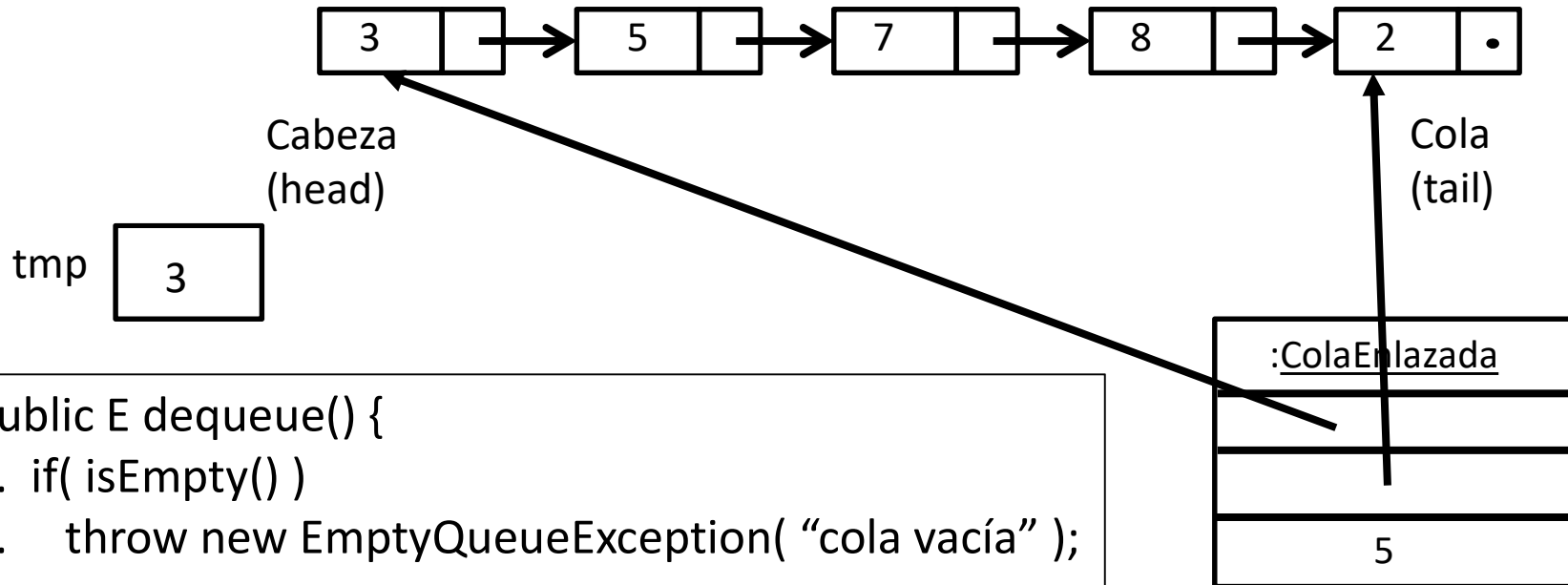


Quiero ejecutar

`q.dequeue();`

Veamos la traza.

Cola: desencolar



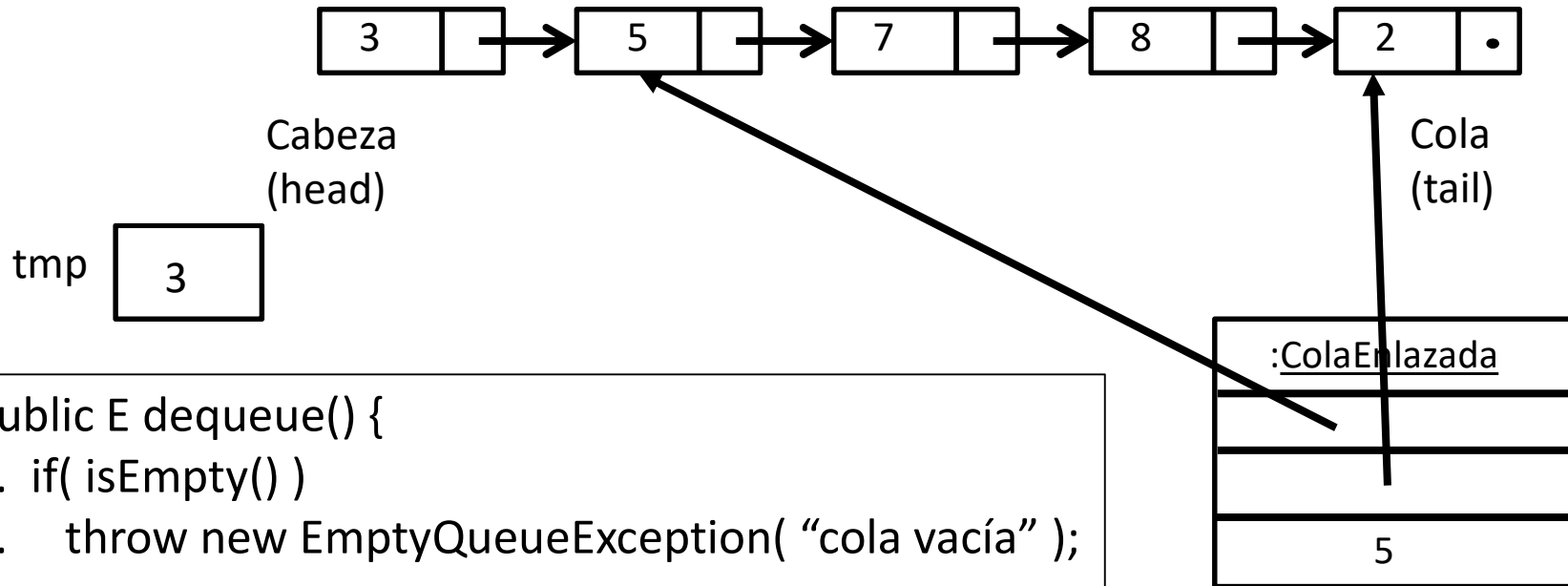
```
public E dequeue() {  
1. if( isEmpty() )  
2.   throw new EmptyQueueException( "cola vacía" );  
  
3. E tmp = head.getElemento();  
  
}
```

Quiero ejecutar

q.dequeue();

Veamos la traza.

Cola: desencolar



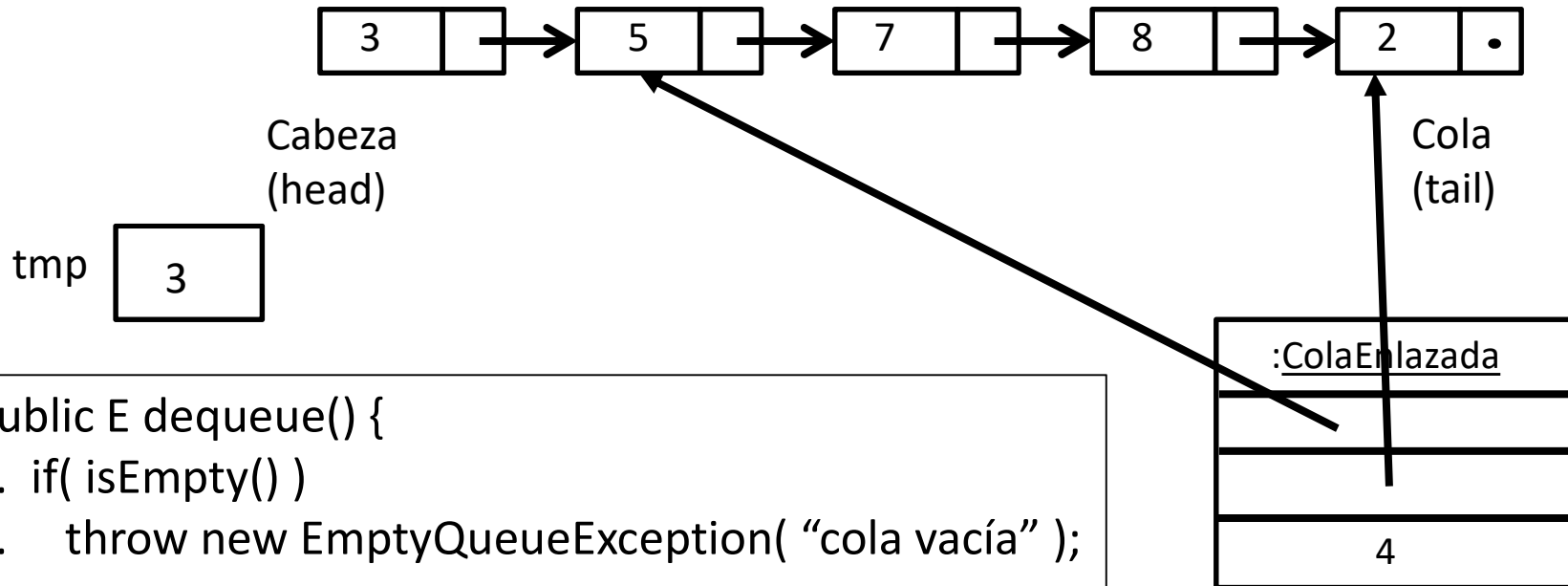
```
public E dequeue() {  
1. if( isEmpty() )  
2.   throw new EmptyQueueException( "cola vacía" );  
  
3. E tmp = head.getElemento();  
4. head = head.getNext();  
  
}
```

Quiero ejecutar

q.dequeue();

Veamos la traza.

Cola: desencolar



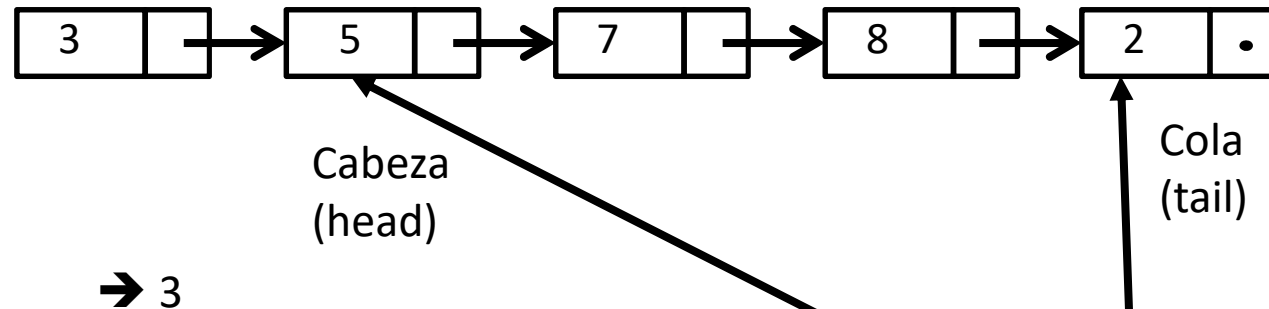
```
public E dequeue() {  
  1. if( isEmpty() )  
  2.   throw new EmptyQueueException( "cola vacía" );  
  
  3. E tmp = head.getElemento();  
  4. head = head.getNext();  
  5. tamaño --;  
  
}
```

Quiero ejecutar

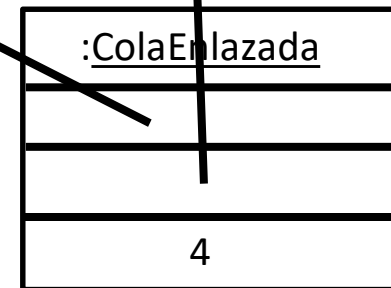
q.dequeue();

Veamos la traza.

Cola: desencolar

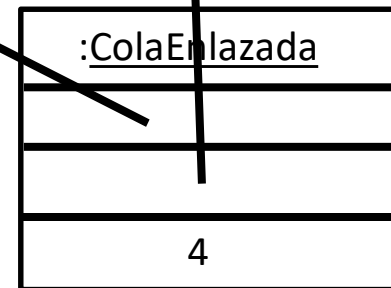
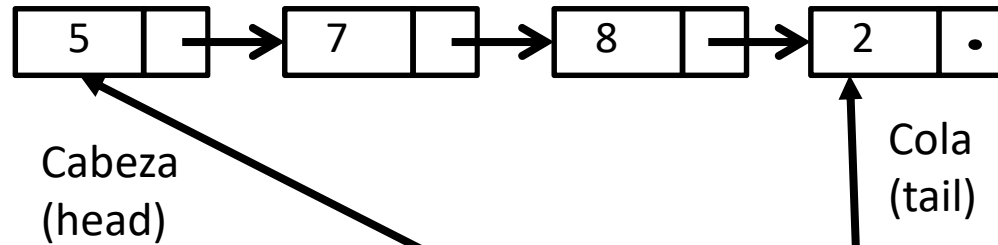


```
public E dequeue() {  
  1. if( isEmpty() )  
  2.   throw new EmptyQueueException( "cola vacía" );  
  
  3. E tmp = head.getElemento();  
  4. head = head.getNext();  
  5. tamaño --;  
  6. if( head == null ) tail = null;  
  7. return tmp;  
}
```



Se destruye la variable local tmp,
Se retorna el 3
El nodo del 3 queda en el heap como
Objeto inalcanzable
Y lo reclama el recolector de basura.

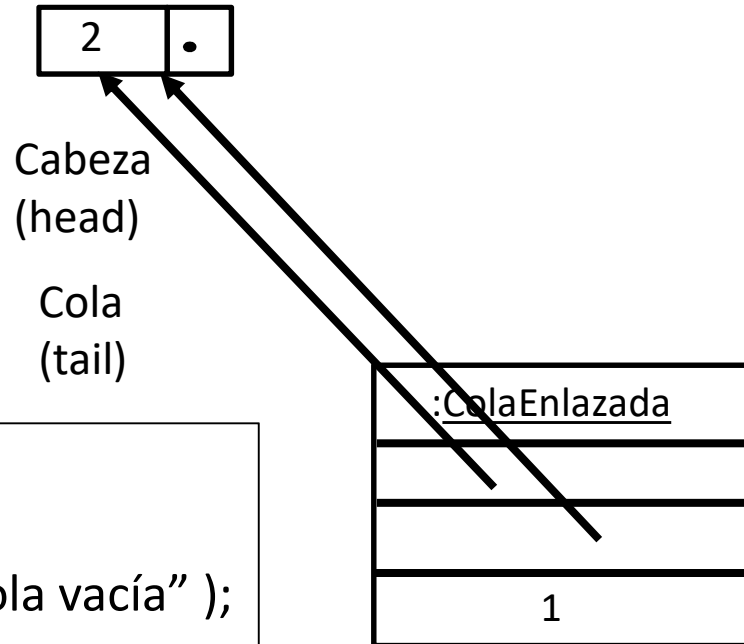
Cola: desencolar



Esta es la cola ahora.

```
public E dequeue() {  
  1. if( isEmpty() )  
  2.   throw new EmptyQueueException( "cola vacía" );  
  
  3. E tmp = head.getElemento();  
  4. head = head.getNext();  
  5. tamaño --;  
  6. if( head == null ) tail = null;  
  7. return tmp;  
}
```

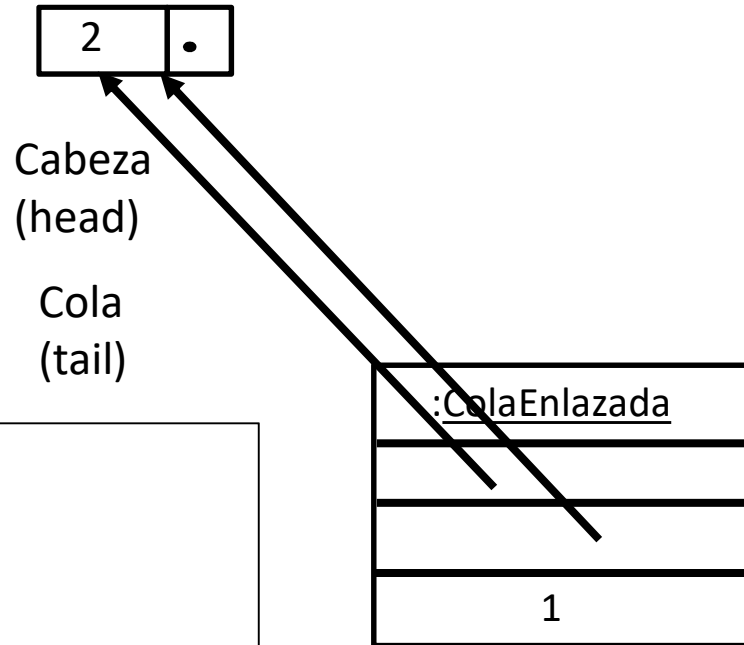
Cola: desencolar cuando la $q.size() = 1$



```
public E dequeue() {  
  1. if( isEmpty() )  
  2.   throw new EmptyQueueException( "cola vacía" );  
  
  3. E tmp = head.getElemento();  
  4. head = head.getNext();  
  5. tamaño --;  
  6. if( head == null ) tail = null;  
  7. return tmp;  
}
```

Ahora head y tail apuntan al mismo nodo.

Cola: desencolar cuando la $q.size() = 1$

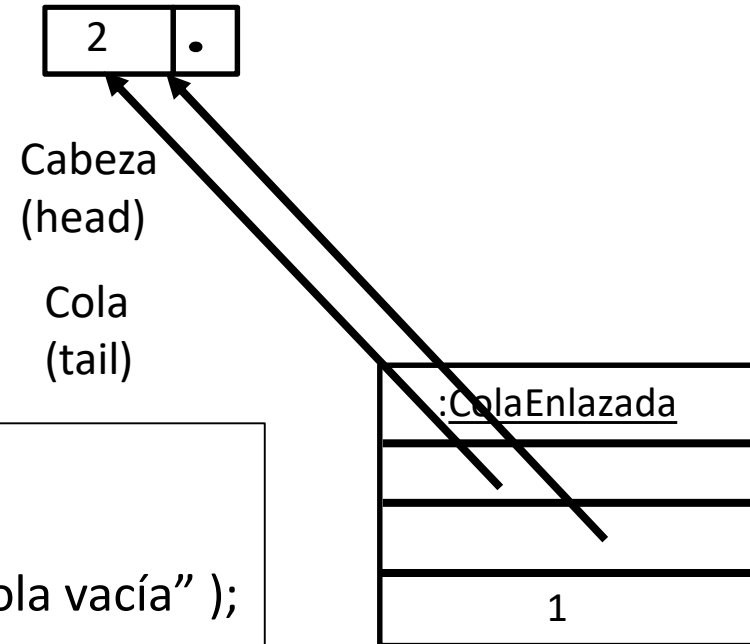


Veamos la traza.

```
public E dequeue() {
```

```
}
```

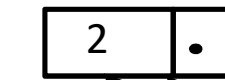
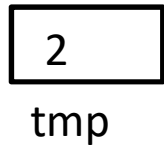
Cola: desencolar cuando la $q.size() = 1$



```
public E dequeue() {  
1. if( isEmpty() )  
2.   throw new EmptyQueueException( "cola vacía" );  
  
}
```

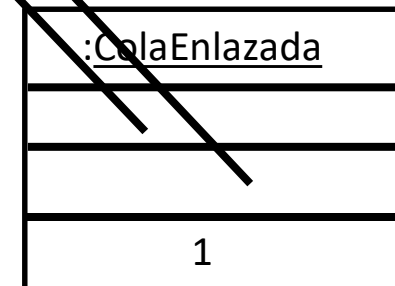
Como tamaño es 1, no se lanza la excepción.

Cola: desencolar cuando la `q.size() = 1`



Cabeza
(head)

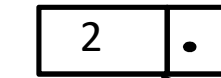
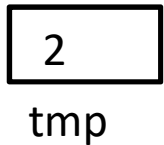
Cola
(tail)



Se crea una variable
local tmp y se asigna 2

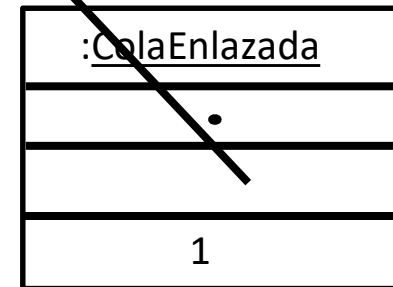
```
public E dequeue() {  
1. If ( isEmpty() )  
2.  throw new EmptyQueueException( "cola vacía" );  
  
3. E tmp = head.getElemento();  
  
}
```

Cola: desencolar cuando la q.size() = 1



Cabeza
(head)

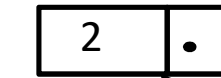
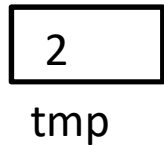
Cola
(tail)



Como el siguiente de
head es null, head toma
el valor null

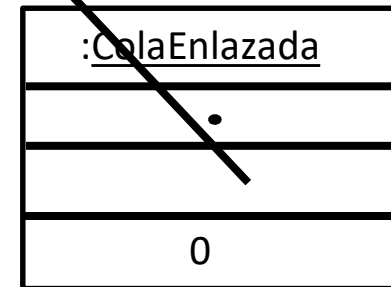
```
public E dequeue() {  
1. if ( isEmpty() )  
2.   throw new EmptyQueueException( "cola vacía" );  
  
3. E tmp = head.getElemento();  
4. head = head.getNext();  
  
}
```

Cola: desencolar cuando la $q.size() = 1$



Cabeza
(head)

Cola
(tail)



Decrementamos
tamaño a 0

```
public E dequeue() {  
1. if( isEmpty() )  
2.   throw new EmptyQueueException( "cola vacía" );  
  
3. E tmp = head.getElemento();  
4. head = head.getNext();  
5. tamaño --;  
  
}
```

Cola: desencolar cuando la `q.size() = 1`

2

tmp

2 •

Cabeza
(head)

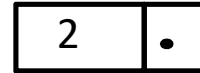
Cola
(tail)

<u>:ColaEnlazada</u>
•
•
0

Como tamaño es 0,
seteamos tail a null

```
public E dequeue() {  
1. if( isEmpty() )  
2.   throw new EmptyQueueException( "cola vacía" );  
  
3. E tmp = head.getElemento();  
4. head = head.getNext();  
5. tamaño --;  
6. if( head == null ) tail = null;  
  
}
```


Cola: desencolar cuando la `q.size() = 1`



→2

```
public E dequeue() {  
  1. if( isEmpty() )  
  2.   throw new EmptyQueueException( "cola vacía" );  
  
  3. E tmp = head.getElemento();  
  4. head = head.getNext();  
  5. tamaño --;  
  6. if( head == null ) tail = null;  
  7. return tmp;  
}
```

<u>:ColaEnlazada</u>
•
•
0

Se retorna el 2, se destruye la variable local tmp, el nodo queda incalcanzable en el heap y se lo lleva el recolector de basura.

Cola: desencolar cuando `q.size() = 0` y `q.isEmpty() = true`

Ahora la cola está vacía.
¿Qué ocurrirá ahora con
la ejecución de
`dequeue()`?

<u>:ColaEnlazada</u>
•
•
0

```
public E dequeue() {
```

```
}
```

Cola: desencolar cuando $q.size() = 0$ y $q.isEmpty() = true$

Como tamaño es 0, dequeue() termina con error.
No se devuelve nada y se lanza una excepción de tipo EmptyQueueException.

<u>:ColaEnlazada</u>
•
•
0

```
public E dequeue() {  
1. if( isEmpty() )  
2.   throw new EmptyQueueException( "cola vacía" );  
  
}
```

Ejemplo: Insertar los elementos 1, 2, 3, 4 en una cola y luego mostrar todos los elementos de la cola.

```
public class App
{
    public static void main( String [] args ) {
        try {
            Queue<Integer> q = new ColaEnlazada<Integer>();

            for( int i=1; i<=4; i++)
                q.enqueue( i );
            while( !q.isEmpty() )
                System.out.println( q.dequeue() );
        } catch( EmptyQueueException e ) {
            e.printStackTrace();
        }
    }
}
```

Bibliografía

- Goodrich & Tamassia, Data Structures and Algorithms in Java, 4th edition, John Wiley & Sons, 2006, Capítulo 5
- Se puede profundizar la parte de nodos enlazados en la Sección 3.2 del libro